King Khalid University,

Department of Information Systems, Applied College, Mahayil Asir

Diploma Programme in Information Systems

Applied Project

Wajhati Saudiya: Smart Tourism Application in the Kingdom of Saudi Arabia

Submitted By:

Student Name	Student ID
Saleha Ahmed Al-Hilali	446813061
Abrar Ahmed Al-Asmari	446817787
Amjad Fahad	446815639
Razan Ali Al-Asmari	446808078
Fatimah Amer Hassan Al-Barqi	446815558
Layla Muhammad Ibrahim	446807557
Jory Yahya Mohammed	445805103
Noora Ali	446808449

Supervised By: Dr. Khalid Mahmood Ali Asghar

Abstract

This system, Wajhati Saudiya, provides a unified platform for domestic tourism planning inside the Kingdom of Saudi Arabia. The proposed system is characterized by organized destination content, user account support, favorite management, destination reviews, interactive browsing, map-based exploration, and smart itinerary generation through both rule-based and optional AI-assisted recommendation logic.

The system allows users to create and review a travel suggestion before confirming and saving it, which increases control and improves usability. The application also provides an administration panel for managing destinations, attractions, users, and AI recommendation settings. This project benefits from current technological development in web systems by offering a practical platform that simplifies trip planning and makes travel information easier to access and use.

Acknowledgment

We want to thank everyone who making it possible for students like us. And we would like to express our deepest gratitude to "Allah" firstly ,

And then to Dr. Khalid Mahmoud, our project supervisor, for his invaluable advices, guidance and his enormous patience throughout the development of the research, and useful critiques of this research work .

We would like to express our gratitude towards our parents for their kind cooperation and encouragement which helped us in finishing this project. And a special thanks to everyone with whom we have reached this stage, especially the faculty members in the college whom we met during our studies.

Committee Report

We certify that we read this graduation project report as examining committee, examined the students' project in its content and that in our opinion it is adequate as a report document for Diploma in Information Systems.

Supervisor: _____ Signature: _____ Date: ___ / ___ / _____

Examiner 1: _____ Signature: _____ Date: ___ / ___ / _____

Examiner 2: _____ Signature: _____ Date: ___ / ___ / _____

Table of Contents

Abstract 2
Acknowledgment 3
Committee Report 4
Chapter 1 8
Introduction 8
1.1 Introduction 9
1.2 Previous Work 9
1.3 Problem Statement 10
1.4 Scope 10
1.5 Objectives 11
1.6 Advantages 11
1.7 Disadvantages 12
1.8 Software Requirements 12
1.9 Hardware Requirements 12
1.10 Software Methodology 13
1.11 Project Plan 13
Chapter 2 15
Literature Review 15
2.1 Tourism Information Systems 16
2.2 Itinerary Planning Systems 16
2.3 Personalization and Recommendation 16
2.4 Web-Based Academic Information Systems 17
2.5 Comparative Positioning of Wajhati Saudiya 18
Chapter 3 19
System Analysis 19
3.1 Stakeholders and Actors 20
3.2 Functional Requirements 20
3.3 Non-Functional Requirements 21
3.4 Business Rules and Assumptions 21
3.5 Use Case Analysis 22
3.6 Use Case Narrative: Register 24

3.7 Use Case Narrative: Login 24
3.8 Use Case Narrative: Generate and Save Itinerary 25
3.9 Use Case Narrative: Administer AI Settings 25
Chapter 4 26
System Design 26
4.1 Architectural Overview 27
4.2 Data and Object Design 28
4.3 Recommendation Design 28
4.4 Security and Validation Design 28
Authentication module 33
Destination browsing module 33
Itinerary planning module 34
Profile personalization module 34
Administration module 35
API module 35
Chapter 5 37
Database and Interfaces 37
5.1 Implementation Overview 38
5.2 Detailed Module Realization 38
5.3 Interface Evaluation 39
5.4 Testing Strategy and Results 39
5.5 Evaluation Against Objectives 39
5.6 Limitations 40
5.7 Future Enhancements 40
5.8 Conclusion 41
Chapter 6 48
Database Design 48
6.1 Database Design Overview 49
6.2 Entity and Attribute Design Rationale 49
6.3 Relationship Design 50
6.4 Evaluation of the Current Database Choice 50
6.5 Database Design Summary 51
Chapter 7 52

User Interface and Conclusion 52
7.1 User Interface Overview 53
7.2 Main Interface Screens 53
7.3 Interface Design Evaluation 54
7.4 Final Conclusion 54
Appendix A: API Summary 56
Appendix B: Data Entity Summary 56
Appendix C: Live Website Interface Images 56
Appendix D: Important Code Snippets 73
D.1 Application Entry Point 73
D.2 Application Factory 73
D.3 Destination Matching Logic 74
D.4 API Itinerary Generation Endpoint 74
D.5 AI Settings Access 75

Chapter 1

Introduction

1.1 Introduction

This system, Wajhati Saudiya, provides a unified platform for domestic tourism planning across destinations in Saudi Arabia. The proposed system is characterized by organized destination content, easy browsing, user accounts, review and favorite features, saved itineraries, map-based destination exploration, and itinerary generation through a smart recommendation process.

The system helps the user discover destinations, review their details, and generate a practical travel plan based on budget, duration, trip type, interests, and optional profile information. The project saves time and effort through the management of destinations, activities, and trip planning in one web application.

This comes as a result of the continuous technological developments that enrich society with digital tools and programs that simplify daily life and decision-making. The project therefore reflects both technical need and practical user demand.

1.2 Previous Work

Many tourism systems provide destination information, city guides, or attraction listings through websites and mobile applications. These systems are useful for presenting tourism information, but they do not always provide personalized itinerary generation, user review workflows, or integrated administration features in one platform.

Some route and travel planning systems allow users to define preferences such as destination, time, and interests, then receive a suggested schedule. However, many of these

systems depend on external services or large datasets and are not designed as lightweight academic web applications focused on local Saudi tourism content.

1.3 Problem Statement

The core problem addressed by this project is that domestic tourism planning is often inefficient when destination information, budget expectations, activity options, and user preferences are not centralized. Many potential travelers know the city they want to visit, but they do not know which attractions fit their available time, which options are likely to stay inside budget, or how to transform scattered information into a coherent day-by-day plan.

A second problem is the absence of lightweight personalization in many basic tourism information systems. Generic destination listings are helpful, but they do not account for user interests, previous preferences, or personal context. In practice, this means that two travelers with very different goals can receive the same list of places even when the better experience would be to produce a tailored itinerary structure.

The system therefore addresses both an information problem and a decision-support problem. It centralizes destination content and then applies ranking and generation logic to help the user move from raw tourism data to an actionable plan. This transition from browsing to planning is the main academic contribution of the project.

1.4 Scope

Because web applications have become widely used and accessible from different places and devices, this project was developed as a web-based system. This makes communication with the system easier, faster, and available from any location through an internet browser.

The current project scope includes local deployment, destination browsing, bilingual user interface support, registration and login, profile personalization, favorites, reviews, saved itineraries, map-based viewing, administrative data entry, and optional AI recommendation configuration. These are the functions that are clearly implemented in the repository and available for demonstration.

The scope does not include online booking, payment, commercial travel inventory, live routing optimization, or production deployment hardening. These boundaries are appropriate for a diploma-level applied project.

1.5 Objectives

The overall aim of the project is to design and implement a smart domestic tourism planning system that helps users discover destinations in Saudi Arabia and generate practical itineraries based on preferences and constraints.

The detailed objectives are to provide searchable destination information, support account-based interaction, store favorites and reviews, generate itinerary suggestions from user input, preserve saved itineraries, expose selected features through a JSON API, and provide administration interfaces for content and AI settings management.

An additional objective is educational. The project is intended to demonstrate how a university team can organize a complete software system using a monolithic but layered architecture, a relational data model, maintainable route structure, reusable service logic, and a documented testing approach.

1.6 Advantages

The system is user-friendly because the graphical web interface allows the user to use the system easily. The application combines several features in one place, including browsing, favorites, reviews, map view, itinerary generation, and administration.

It also supports both Arabic and English in major interface elements, which improves accessibility for different users. The system provides smart planning support while still allowing the user to confirm or regenerate the suggestion before it is stored.

1.7 Disadvantages

The system depends on the currently stored destination dataset, so the quality of suggestions is limited by available data. The AI service depends on external configuration and internet connectivity when enabled.

The database design currently uses SQLite and automatic table creation, which is suitable for development but not ideal for large-scale deployment. The project is a prototype and not a commercial tourism platform.

1.8 Software Requirements

The software tools required to develop and run the system include Python, Flask, SQLAlchemy, SQLite, Jinja templates, Tailwind CSS, and Leaflet. The project also supports optional Gemini API integration for AI-assisted itinerary generation.

These technologies were selected because they provide a practical balance between simplicity, maintainability, and feature coverage for a university project environment.

1.9 Hardware Requirements

The hardware requirements of the system are modest because it targets local development and academic demonstration. A standard personal computer with sufficient RAM, storage, and internet access is adequate to run the application and demonstrate its main workflows.

This low hardware requirement makes the project suitable for university environments where reproducibility and accessibility are important.

1.10 Software Methodology

The observed structure of the repository reflects an iterative methodology similar to agile development. Functionality is organized in increments such as authentication, browsing, itinerary generation, profile handling, administration, and testing.

This methodology is suitable for the project because the application contains both user-facing and technical workflows. It supports gradual improvement, easier testing, and frequent feature extension during development.

1.11 Project Plan

The project plan follows a practical sequence: problem identification, requirement extraction, model design, route and template implementation, recommendation service development, administration workflow integration, testing, and documentation. Each stage corresponds to evidence visible in the repository, including models, routes, templates, tests, and report drafts.

From a management perspective, this ordering is sound because it establishes the data model and user flow before optimizing peripheral features. Once those foundations are stable,

secondary concerns such as personalization, API access, and AI configuration can be integrated without destabilizing the core domain entities.

Table 1.1 Technology Stack

Technology	Role in the System
Python 3	Primary implementation language for the application, scripts, and tests.
Flask	HTTP routing, request processing, template rendering, and blueprint organization.
SQLAlchemy / Flask-SQLAlchemy	Relational data modeling, persistence, and query access.
SQLite	Local relational database used for the project demonstration environment.
Flask-Login	Session-based authentication, login state, and protected views.
Jinja Templates	Server-rendered pages for the web interface.
Leaflet	Map presentation for destination location display.
Gemini API (optional)	Configurable AI-assisted itinerary generation path.

Chapter 2

Literature Review

2.1 Tourism Information Systems

Tourism information systems typically begin with directory-style functionality: listings of destinations, descriptive content, maps, media, and contact information. These systems are valuable because they digitize access to destination knowledge, yet they often stop at information provision. They do not always help the user decide how to combine places into a feasible trip.

For Wajhati Saudiya, this category is an important baseline rather than a final target. The project includes browsing and destination details, but it extends beyond them by adding account-based interaction, reviews, favorites, and itinerary generation. In other words, the project transforms a passive catalog into a more active decision-support system.

2.2 Itinerary Planning Systems

Itinerary planning systems usually accept traveler constraints such as destination, available time, and budget, then translate these inputs into an ordered plan. Their value lies in reducing planning effort and helping users see how separate attractions can form a coherent trip. However, many such systems depend on complex external services or broad commercial datasets that are not suitable for a compact academic implementation.

Wajhati Saudiya adopts the itinerary planning concept but keeps the implementation lightweight and explainable. The system uses direct destination matching, predictable ranking criteria, and a day-based allocation model. This makes the recommendation behavior easier to understand, test, and document, which is important in an educational context.

2.3 Personalization and Recommendation

Personalization in tourism software can range from simple profile tags to advanced learning-based recommendation pipelines. In many educational projects, heavy machine learning is impractical because it requires large datasets, extensive tuning, and significant evaluation effort. A simpler approach is to combine profile metadata with explicit user preferences and clear ranking rules.

That is the path taken here. Wajhati Saudiya blends city matching, category matching, budget fit, keyword overlap, and favorite tags. The result is not a black-box prediction engine; it is a controllable recommendation service that can be explained in a viva or committee review. The optional AI enhancement does not replace this logic. Instead, it operates as a configurable layer that can enrich itinerary composition while still falling back to rule-based planning when necessary.

2.4 Web-Based Academic Information Systems

A strong applied project must demonstrate more than a collection of pages. It should show data integrity, controlled navigation, permission boundaries, maintainable code organization, and at least a basic testing strategy. Many student systems provide interface prototypes but do not include the back-office and data relationships necessary to qualify as robust information systems.

This project is stronger than a pure interface prototype because it includes concrete persistence, authenticated workflows, administrative functions, API endpoints, and automated tests. These qualities are important when comparing it with simpler course projects that concentrate only on visual presentation or database schemas without full application behavior.

2.5 Comparative Positioning of Wajhati Saudiya

The comparative strength of Wajhati Saudiya lies in balance. It combines tourism browsing, guided planning, user-generated interaction, personalization, and administration without becoming dependent on enterprise-scale infrastructure. That balance makes it particularly suitable for a diploma project because it is large enough to demonstrate depth but still small enough to be reviewed comprehensively.

The system also illustrates a pragmatic design philosophy. Rather than promising large-scale predictive intelligence, it guarantees that a useful result can still be produced in fallback mode. In software engineering terms, that is a resilience feature. In academic terms, it shows appropriate scoping and responsible system design.

Table 2.1 Comparative View of Related System Types

System Type	Typical Strength	Common Limitation	Wajhati Saudiya Position
Destination directories	Rich descriptive browsing content.	Limited decision support.	Extends browsing into itinerary planning.
Trip planners	Transforms preferences into schedules.	May depend on heavy external services.	Uses lightweight and explainable planning logic.
Map explorers	Good geographic visualization.	Weak account-based personalization.	Combines maps with reviews, favorites, and saved trips.
Student prototypes	Fast to demonstrate.	Often lack persistence and admin control.	Includes database-backed workflows and administration.

In summary, the literature and product landscape show that Wajhati Saudiya occupies a realistic middle ground: richer than a destination catalog, more explainable than a heavy AI travel planner, and more complete than a purely presentational student interface.

Chapter 3

System Analysis

3.1 Stakeholders and Actors

Three actor categories are visible in the implemented system: visitor, authenticated user, and administrator. The visitor can browse public information, inspect destinations, use the map, and create an account. The authenticated user inherits those capabilities and gains personalized interaction such as reviews, favorites, profile updates, itinerary generation, and saved trips. The administrator is responsible for content and configuration governance.

This role separation is important because it prevents the report from presenting the application as a single undifferentiated interface. In reality, the information needs and permissions of each actor are different. A mature information system must encode those differences in both route access and data behavior.

3.2 Functional Requirements

The system shall allow a visitor to register a new account using name, email, and password. The system shall allow login using email or username and shall maintain authenticated session state after successful sign-in. The system shall support browsing destinations and filtering by city or category.

The system shall allow an authenticated user to manage a preference profile, submit reviews, toggle favorites, generate itinerary suggestions, preview a generated itinerary before saving, confirm and save an itinerary, display saved itineraries, and delete owned itineraries. The system shall also expose selected features through JSON API routes.

The system shall allow an administrator to manage destinations, attractions, users, and AI configuration settings. These functional requirements are not theoretical; they correspond to explicit route groups and model relationships in the repository.

3.3 Non-Functional Requirements

The application should be understandable, maintainable, and demonstrable in a university environment. This implies clear route separation, straightforward model design, and limited infrastructure complexity. The use of Flask blueprints and modular files supports this requirement.

The application should validate key user inputs before database writes or itinerary generation. It should preserve data consistently in a relational schema and enforce basic constraints such as unique user email addresses and unique user-destination favorites. It should degrade gracefully when the optional AI path is unavailable by falling back to the deterministic recommendation flow.

Additional non-functional requirements include bilingual support, reasonable response clarity, security boundaries for administration pages, and the presence of automated tests for critical behaviors. These do not replace future production hardening, but they represent meaningful quality attributes for the current project stage.

3.4 Business Rules and Assumptions

The implementation encodes several business rules that are directly relevant to system analysis. A favorite relationship must be unique per user and destination. Duration days are

constrained in the API layer to a short range suitable for local tourism planning. Budget values must be numeric and positive. Reviews are associated with both a user and a destination.

Itinerary generation follows an assumption that a short, practical plan is preferable to an exhaustive list of all available places. Therefore, the system ranks candidate destinations and then allocates selected items across trip days. If the estimated total cost exceeds the stated budget, the rule-based generator scales per-item cost estimates proportionally rather than producing an unusable result.

Another important assumption is that profile metadata should influence recommendations without preventing generation. In other words, age range, gender, and favorite tags refine the result, but their absence does not block the workflow. This keeps the system inclusive and reduces empty-result scenarios.

3.5 Use Case Analysis

The main use cases are registration, login, browse destinations, view destination details, generate itinerary, confirm itinerary, submit review, manage favorites, update profile, and administer system data. Among these, itinerary generation is the central use case because it connects content browsing, profile context, and persistence into one end-to-end business flow.

The use case model also illustrates why the application deserves to be described as a system rather than a set of unrelated pages. Each actor goal maps to a coordinated sequence of validation, query execution, service logic, rendering, and optional storage. This coordination is what gives the project its information systems character.

Figure 3.1 Use Case Diagram for Wajhati Saudiya

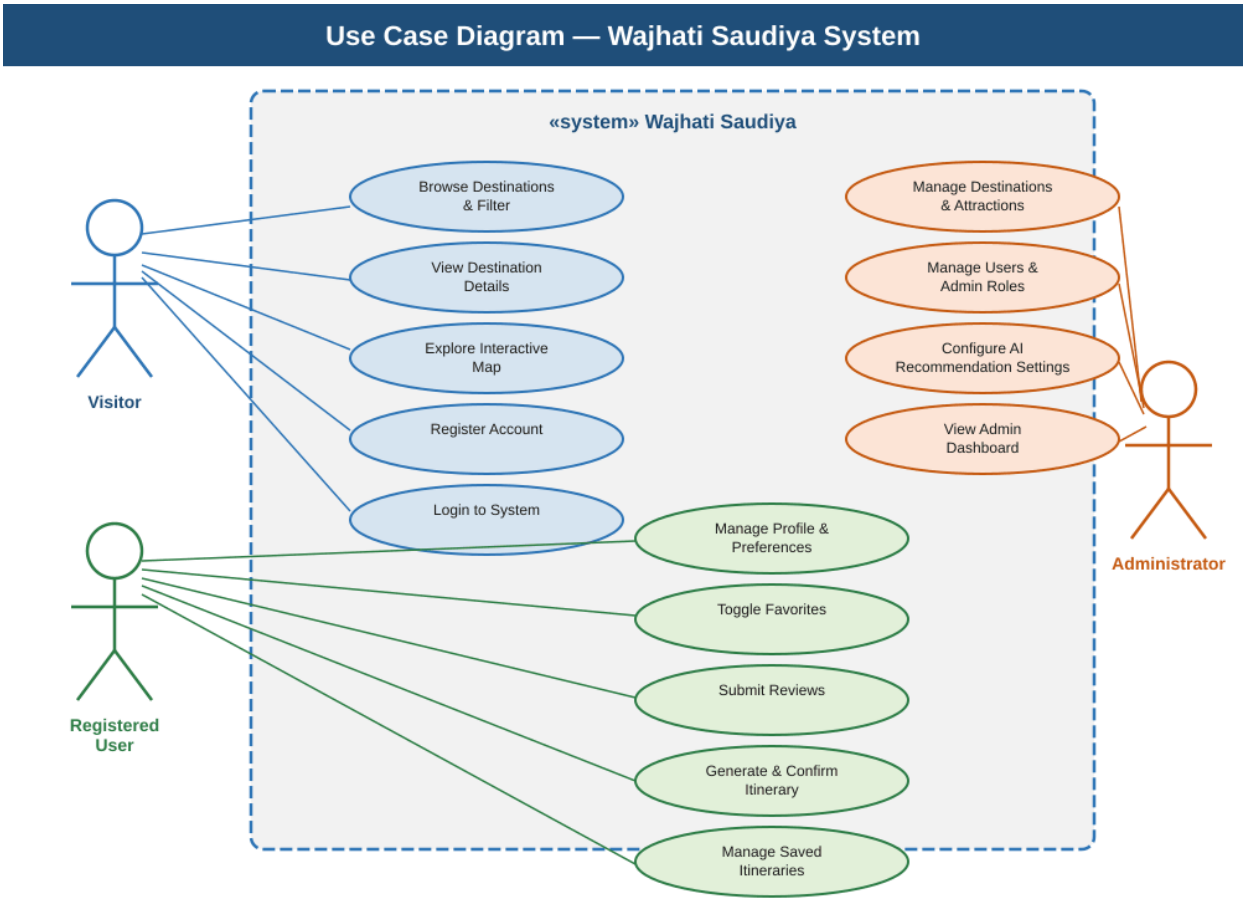


Figure 3.1 shows the major actors and the system services available to each one. The diagram makes it clear that Wajhati Saudiya must handle both public browsing and protected workflows, and those administrative actions are intentionally separated from normal user actions.

Table 3.1 Core Requirement Matrix

Requirement Area	Representative Requirement	Evidence in the Implementation
Authentication	Users can register, log in, and log out securely.	Dedicated authentication blueprint and user model password hashing.
Destination	Users can browse destinations by city	Main routes and destination queries.

management	and category.	
Recommendation	The system generates itinerary suggestions from user inputs.	Recommendation service and itinerary routes/API.
Persistence	Generated itineraries can be stored and retrieved later.	Itinerary and itinerary item models with user ownership.
Administration	Admins can manage system data and AI settings.	Admin-only routes and AppSetting model.

3.6 Use Case Narrative: Register

The registration use case starts when a visitor submits a name, email address, and password. The system checks for completeness, verifies that the email is not already registered, hashes the password, creates the new user record, and redirects the user toward login. This use case is fundamental because it transforms anonymous browsing into identity-based interaction.

From an analysis perspective, this use case is valuable because it crosses interface, validation, persistence, and service boundaries. That multi-layer behavior is exactly the kind of complexity that should be documented in an applied project report.

3.7 Use Case Narrative: Login

The login use case starts when a user enters an identifier and password. The system supports either email or username as the identifier, retrieves the matching user, validates the password hash, opens the authenticated session, and redirects the user safely. This flow demonstrates both usability and session control.

From an analysis perspective, this use case is valuable because it crosses interface, validation, persistence, and service boundaries. That multi-layer behavior is exactly the kind of complexity that should be documented in an applied project report.

3.8 Use Case Narrative: Generate and Save Itinerary

The itinerary use case begins when the authenticated user enters planning constraints. The system validates the input, loads profile context and destinations, computes ranked matches, generates a preview itinerary, and lets the user confirm or regenerate it. Once confirmed, the itinerary and day items are written to the database and become available in the saved itineraries view.

From an analysis perspective, this use case is valuable because it crosses interface, validation, persistence, and service boundaries. That multi-layer behavior is exactly the kind of complexity that should be documented in an applied project report.

3.9 Use Case Narrative: Administer AI Settings

The AI settings use case is initiated by an administrator. The system receives a model name, API key, enablement flag, and prompt text, validates the configuration, stores the values in the AppSetting table, and uses them later at runtime when itinerary generation requests are processed. This use case demonstrates runtime configurability without code modification.

From an analysis perspective, this use case is valuable because it crosses interface, validation, persistence, and service boundaries. That multi-layer behavior is exactly the kind of complexity that should be documented in an applied project report.

Chapter 4

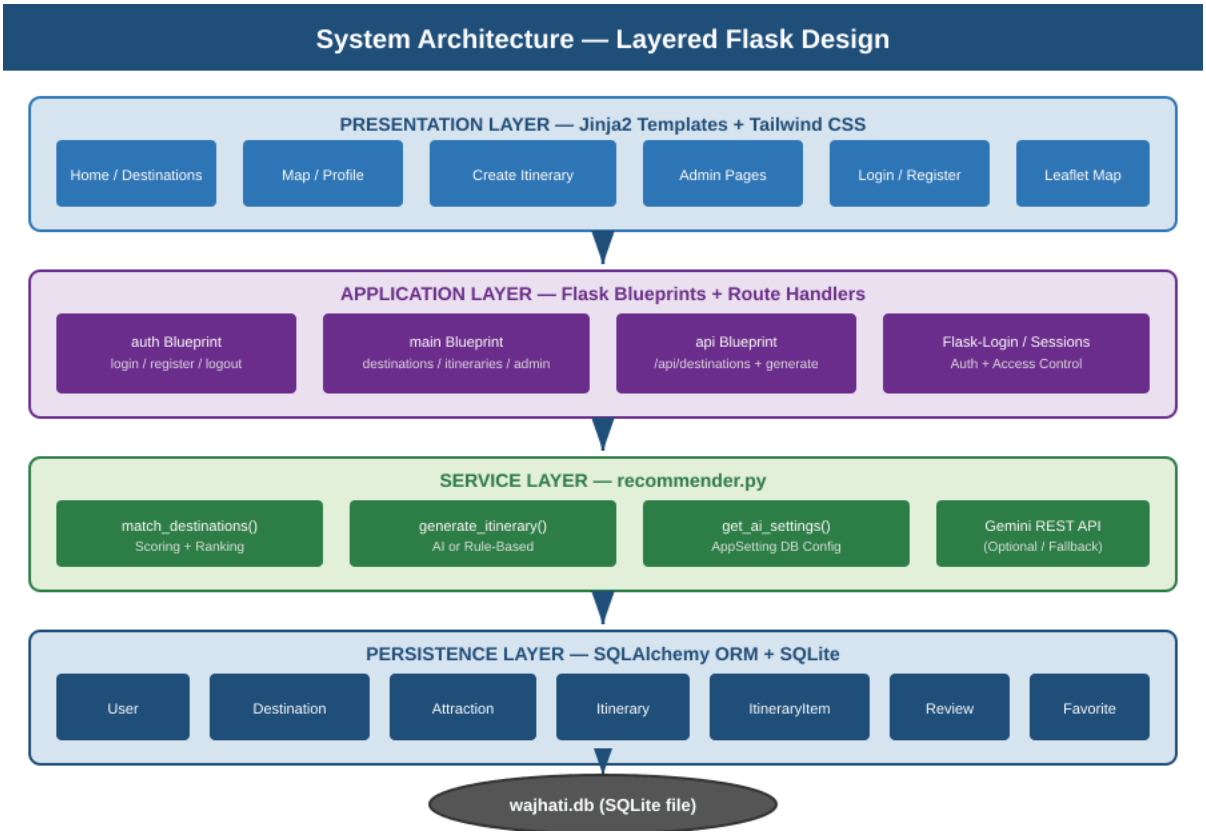
System Design

4.1 Architectural Overview

Wajhati Saudiya follows a layered monolithic architecture. This means the application is deployed as one coherent Flask project while still separating presentation, routing, services, and persistence responsibilities. The design is appropriate for the current size of the system because it avoids unnecessary distributed complexity but still preserves internal structure.

The presentation layer consists of Jinja templates and static assets. The application layer consists of blueprints that define user-facing and API-facing routes. The service layer contains recommendation behavior and optional AI integration. The data layer is defined in SQLAlchemy models that express entities, constraints, and relationships. This decomposition aligns closely with standard software engineering teaching on modular design.

Figure 4.1 System Architecture Diagram: Four-Layer Flask Design



4.2 Data and Object Design

The object model revolves around seven primary domain concepts and one configuration entity. User, Destination, Attraction, Favorite, Review, Itinerary, and ItineraryItem together describe the traveler domain and trip-planning behavior, while AppSetting stores configurable system values that are not tied to a single business object instance.

This model is well chosen because it separates stable content from user interaction. Destinations and attractions represent curated tourism data, while favorites, reviews, and itineraries represent user-generated interaction. The separation prevents semantic overload in the schema and supports clearer route logic.

4.3 Recommendation Design

The recommendation subsystem is designed around progressive decision support. The first step is destination matching, where candidates are scored according to city alignment, budget fit, category overlap, profile favorite tags, and textual overlap with interests. The second step is itinerary generation, where matched destinations are converted into day-based items and a total cost estimate.

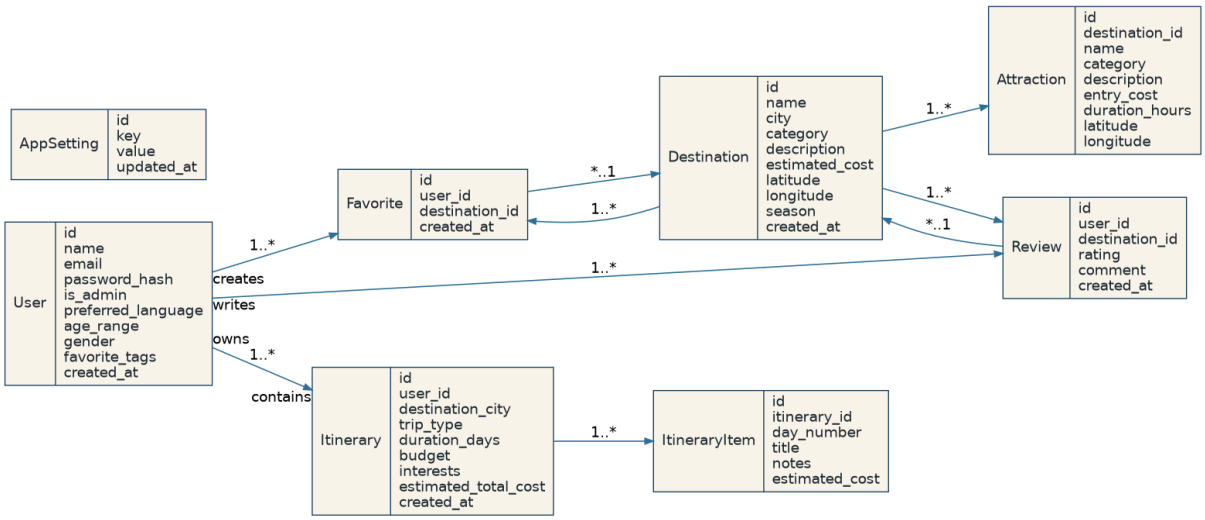
A notable design choice is the fallback mechanism. If AI recommendations are enabled and configured, the system can call an external Gemini model and normalize the resulting JSON. If that path fails for network, validation, or parsing reasons, the service logs a warning and reverts to the internal rule-based generator. This is a robust and academically defensible design because it avoids total workflow failure when an optional dependency is unavailable.

4.4 Security and Validation Design

Security in the current project is pragmatic and route-centered. Passwords are hashed before storage, protected views depend on authenticated session state, and administrative pages check that the current user has administrative privileges. Safe redirect checking is used during authentication to reduce open redirect risk.

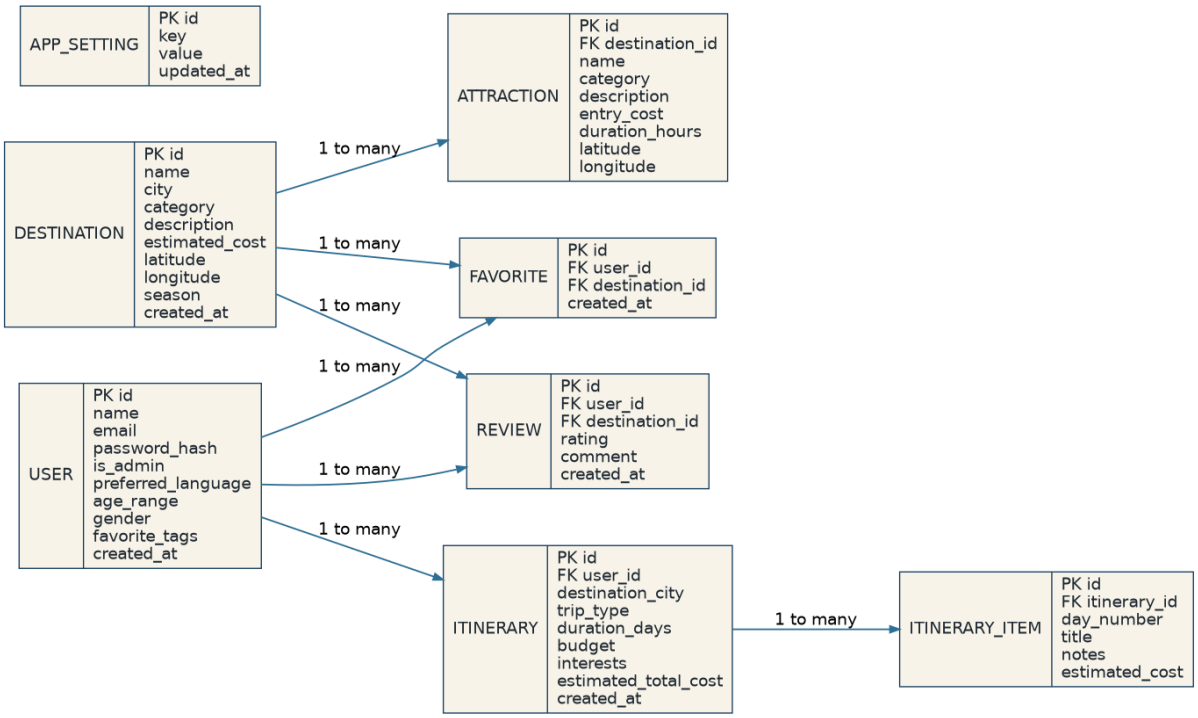
Validation is also layered. Forms and API payloads are checked for required data, allowed value sets, positive numeric ranges, and relationship existence. This helps prevent invalid records and improves the reliability of recommendation output. While the project is not yet production hardened, the existing validation strategy is meaningful and well aligned with the scope of a diploma system.

Figure 4.1 Class Diagram



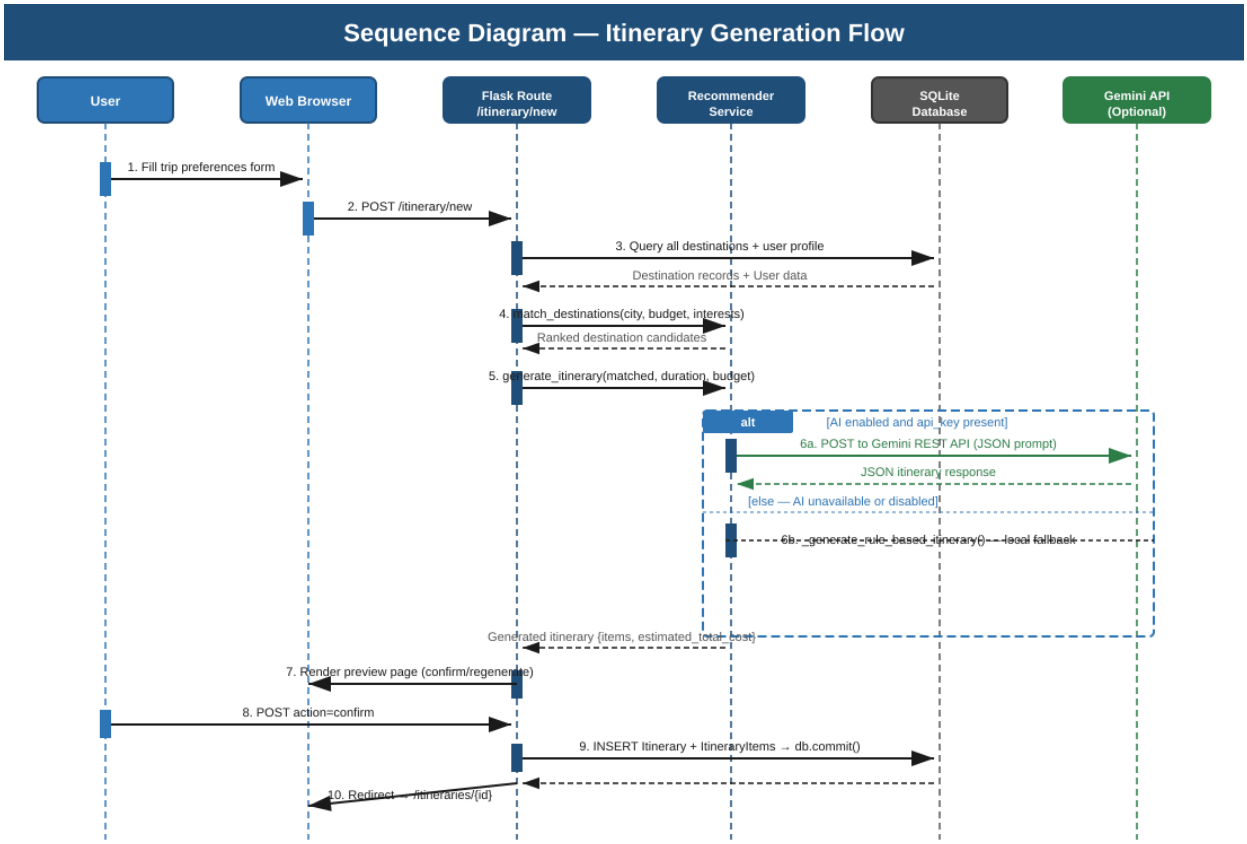
The class diagram emphasizes the strong relationships between account ownership, tourism content, and itinerary persistence. It also makes visible the separate configuration entity used for AI-related settings, which is an implementation detail that matters when discussing extensibility and administration.

Figure 4.2 Entity Relationship Diagram



From a database perspective, the ERD confirms that the schema is normalized around clear subject areas. User-generated interaction is represented through linking entities such as favorites and reviews, while itineraries and itinerary items model trip persistence in a way that preserves day-level structure.

Figure 4.3 Sequence Diagram for Itinerary Generation



The sequence diagram highlights the central runtime path of the project: a user request enters through the web UI, the route loads data and profile context, matching is performed, the itinerary service produces a result, and the preview is returned to the user. The optional AI call is represented as an alternate path rather than a mandatory dependency.

Figure 4.4 — Activity Diagram: Complete Itinerary Planning Flow

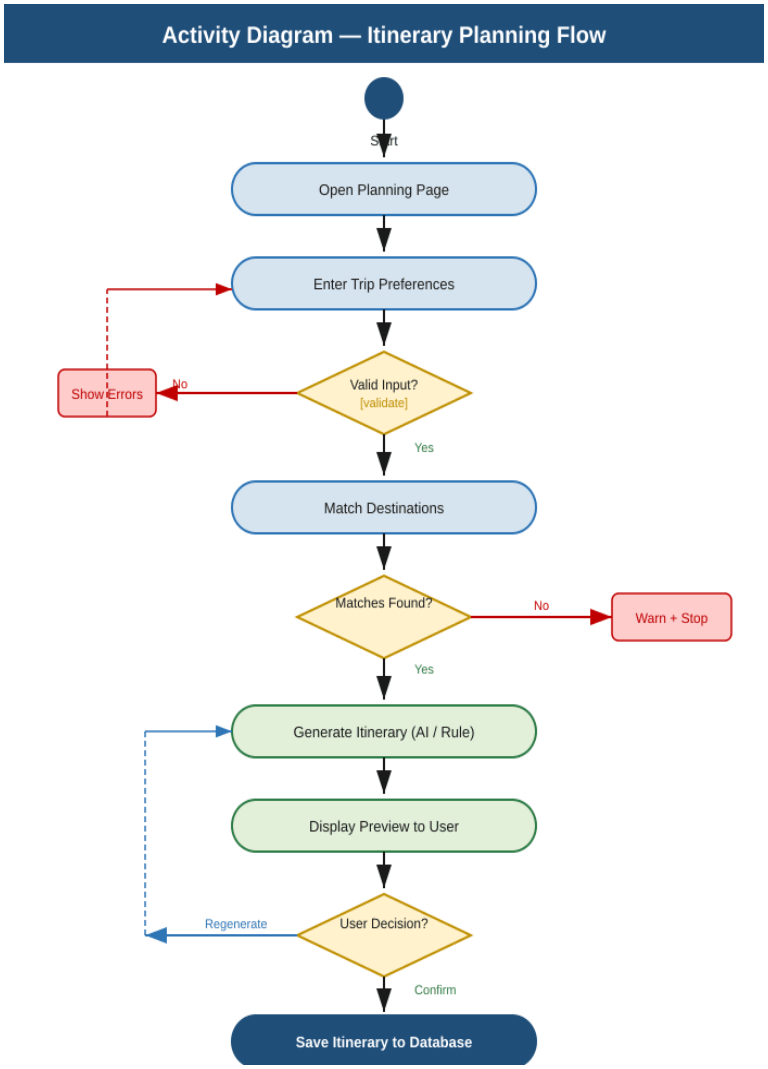


Table 4.1 Activity Description

Step	Activity	Decision / Branch
1	User opens the planning page	—
2	User enters trip preferences (city optional, duration, budget, type, interests)	—
3	System validates inputs	Invalid → show error flash, return to form; Valid → continue
4	System calls match_destinations()	No matches found → show warning, halt; Matches found → continue
5	System calls	AI enabled and key present → call Gemini API; else

	generate_itinerary()	→ rule-based generator
6	Generated itinerary displayed as preview	—
7	User reviews preview	Regenerate → return to step 5; Confirm → save to DB
8	Itinerary and items persisted to database	—
9	User redirected to itinerary detail page	—

Authentication module

The authentication blueprint handles registration, login, logout, and safe redirection. It accepts either an email address or a username during login, which improves usability in a student demonstration environment and reduces avoidable friction during evaluation sessions.

In architectural terms, the authentication module improves separation of concerns because controller logic, validation, domain models, and persistence responsibilities remain distinct. This improves maintainability, makes the code easier to review in a university setting, and creates a clearer path for later enhancement or refactoring.

Destination browsing module

The browsing layer allows visitors and registered users to filter destinations by city and category. The implementation combines SQLAlchemy queries with server-rendered templates so that the user can move from summary exploration to destination-level detail without leaving the application flow.

In architectural terms, the destination browsing module improves separation of concerns because controller logic, validation, domain models, and persistence responsibilities remain

distinct. This improves maintainability, makes the code easier to review in a university setting, and creates a clearer path for later enhancement or refactoring.

Itinerary planning module

The itinerary workflow collects destination city, trip duration, budget, trip type, and interest tags. After validation, the application performs destination matching and itinerary generation, then returns a preview that can be confirmed or regenerated before any data are persisted.

In architectural terms, the itinerary planning module improves separation of concerns because controller logic, validation, domain models, and persistence responsibilities remain distinct. This improves maintainability, makes the code easier to review in a university setting, and creates a clearer path for later enhancement or refactoring.

Profile personalization module

The user profile extends the basic account with age range, gender, and favorite tags. These values are not cosmetic; they become runtime inputs for the recommendation layer and therefore provide a lightweight form of personalization without introducing a complex analytics pipeline.

In architectural terms, the profile personalization module improves separation of concerns because controller logic, validation, domain models, and persistence responsibilities remain distinct. This improves maintainability, makes the code easier to review in a university setting, and creates a clearer path for later enhancement or refactoring.

Administration module

The administration area supports destination management, attraction management, user oversight, and AI configuration. This is academically important because it demonstrates that the system is not only a visitor-facing interface, but also an information system with governance and content maintenance capabilities.

In architectural terms, the administration module improves separation of concerns because controller logic, validation, domain models, and persistence responsibilities remain distinct. This improves maintainability, makes the code easier to review in a university setting, and creates a clearer path for later enhancement or refactoring.

API module

The API blueprint exposes health checking, destination listing, itinerary generation, and review retrieval as JSON services. This allows the same business rules to support future mobile or external clients and illustrates service reusability within a monolithic architecture.

In architectural terms, the api module improves separation of concerns because controller logic, validation, domain models, and persistence responsibilities remain distinct. This improves maintainability, makes the code easier to review in a university setting, and creates a clearer path for later enhancement or refactoring.

Table 4.2 Main Domain Entities

Entity	Purpose	Important Relationships
User	Stores identity, roles, and preference metadata.	Owns itineraries, reviews, and favorites.
Destination	Stores tourism destination details.	Owns attractions and receives

		reviews/favorites.
Attraction	Represents a specific activity linked to a destination.	Belongs to one destination.
Itinerary	Stores a generated trip request and total cost.	Belongs to one user and owns itinerary items.
ItineraryItem	Stores one planned activity within a trip day.	Belongs to one itinerary.
Review	Captures a user rating and comment.	Links a user to a destination.
Favorite	Tracks a user-destination bookmark.	Unique per user and destination.
AppSetting	Stores configurable runtime settings.	Used especially for AI-related configuration.

Chapter 5

Database and Interfaces

5.1 Implementation Overview

The implemented system is organized as a Flask application factory with extensions and blueprints registered in a centralized initialization flow. This structure supports clean application startup, easier test setup, and maintainable route grouping. The `app.py` entry point is intentionally small because the real application behavior lives in reusable package modules under the `wajhati` namespace.

The codebase demonstrates sensible layering for a student project. Models are defined centrally, recommendation behavior is grouped in a service module, and routes are separated by responsibility. This reduces file-level confusion and makes the system easier to explain during a project defense.

5.2 Detailed Module Realization

The authentication module uses password hashing and the Flask-Login session model. The destination module combines listing, detail presentation, and filtering. The itinerary module orchestrates validation, destination matching, itinerary generation, preview rendering, confirmation, and deletion. The API module mirrors key planning features through JSON endpoints so that the project is not tied only to one UI surface.

The recommendation service deserves special attention because it captures the system's smart behavior. It normalizes profile data, computes matching scores, and composes a multi-day result. The AI integration is designed as an enhancement rather than a single point of failure, which is a strong engineering decision because it improves flexibility without weakening baseline reliability.

5.3 Interface Evaluation

The user interface is built as a practical navigation flow rather than a collection of isolated pages. The home page introduces the platform, the preferences page collects planning inputs, the itinerary screen presents the generated result, and the map page provides spatial context for destination exploration. Together, these views show that the application supports a complete user journey from first access to saved travel plan.

From a human-computer interaction perspective, the preview-confirm-regenerate behavior is one of the strongest aspects of the project. It avoids forcing users to accept a generated itinerary blindly and instead frames the recommendation as a suggestion under user control. This improves trust and is appropriate for a planning support system.

5.4 Testing Strategy and Results

The repository includes automated tests for critical recommendation and API flows. This is important because itinerary generation is the most business-critical feature of the application. Tests confirm that destination matching behaves correctly for city-constrained requests and that API responses follow expected success and failure patterns.

Although the current test suite is not exhaustive, it already shows an awareness of quality assurance beyond manual clicking. For a diploma project, this is a meaningful strength because it demonstrates that the team considered repeatability, regression prevention, and behavior verification rather than relying exclusively on screenshots or claims about correctness.

5.5 Evaluation Against Objectives

The implemented system satisfies the major stated objectives. It provides a searchable domestic tourism catalog, account-based interaction, personalized preference capture, favorites, reviews, itinerary generation, itinerary persistence, and API exposure. It also includes administration capabilities that make the platform manageable rather than static.

Not every desired enhancement is complete. The project remains local-development oriented, uses SQLite, and does not yet integrate live maps, routing optimization, or booking services. However, those limitations do not invalidate the success of the current objectives. Instead, they define a reasonable boundary between a completed academic prototype and a future production roadmap.

5.6 Limitations

The destination dataset is limited and suited primarily to demonstration. The quality of generated itineraries will naturally depend on the breadth and quality of available destination and attraction records. A larger real-world deployment would require stronger content management processes and much broader destination coverage.

The AI integration also remains configuration dependent. When enabled, it requires valid credentials and network availability. For that reason, the rule-based fallback remains essential. Additional limitations include the absence of migration tooling, deeper security hardening, and broader automated coverage for every route and edge case.

5.7 Future Enhancements

Future work can proceed in several directions. The system could integrate migration tooling, improve test coverage, expand bilingual content, and connect destination coordinates to

richer route-aware mapping behavior. It could also incorporate stronger administrative analytics and reporting.

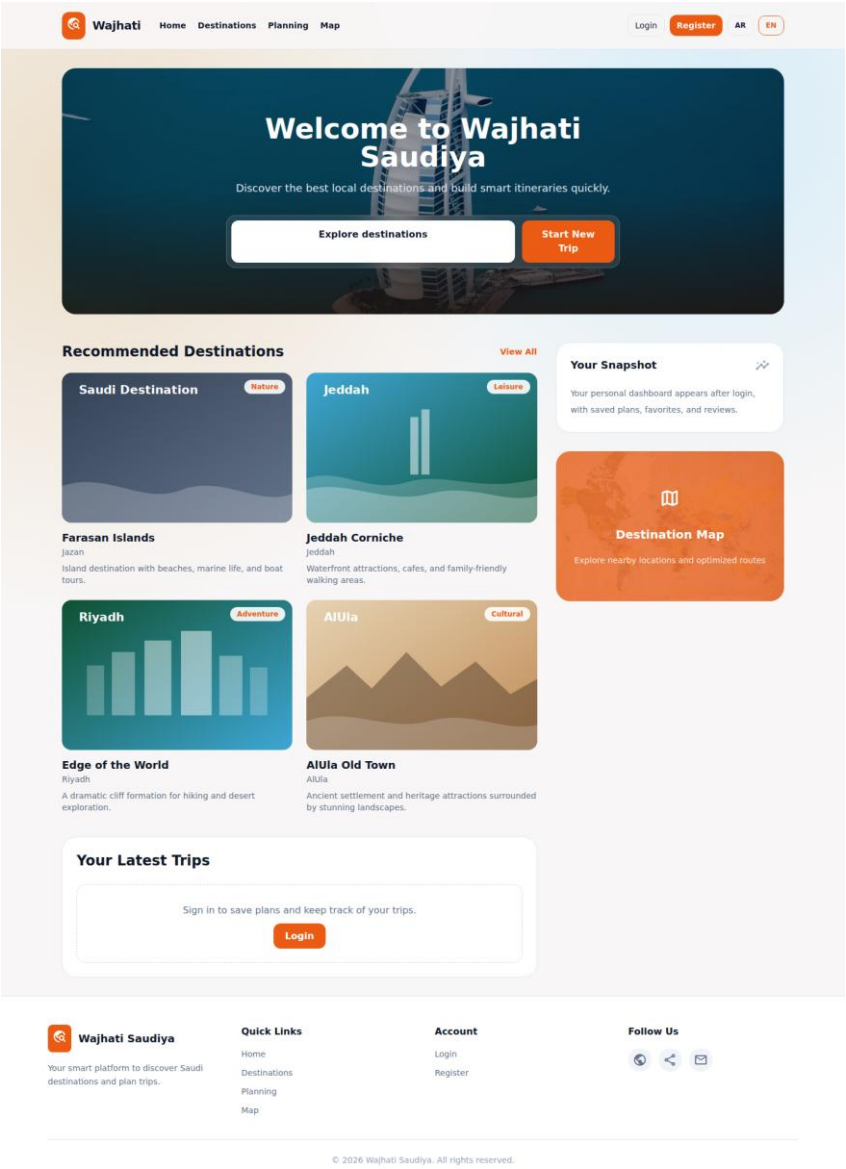
From a recommendation perspective, future development could compare rule-based and AI-generated outputs more systematically, introduce seasonal weighting, and factor attraction-level scheduling into daily itinerary composition. Those enhancements would make the system more realistic while preserving the current architecture as a stable foundation.

5.8 Conclusion

Wajhati Saudiya demonstrates that a domestic tourism planning system can be implemented as a coherent academic information system rather than as a disconnected prototype. The application combines data management, user interaction, planning support, and configurable intelligence in a way that is technically meaningful and easy to present in a university setting.

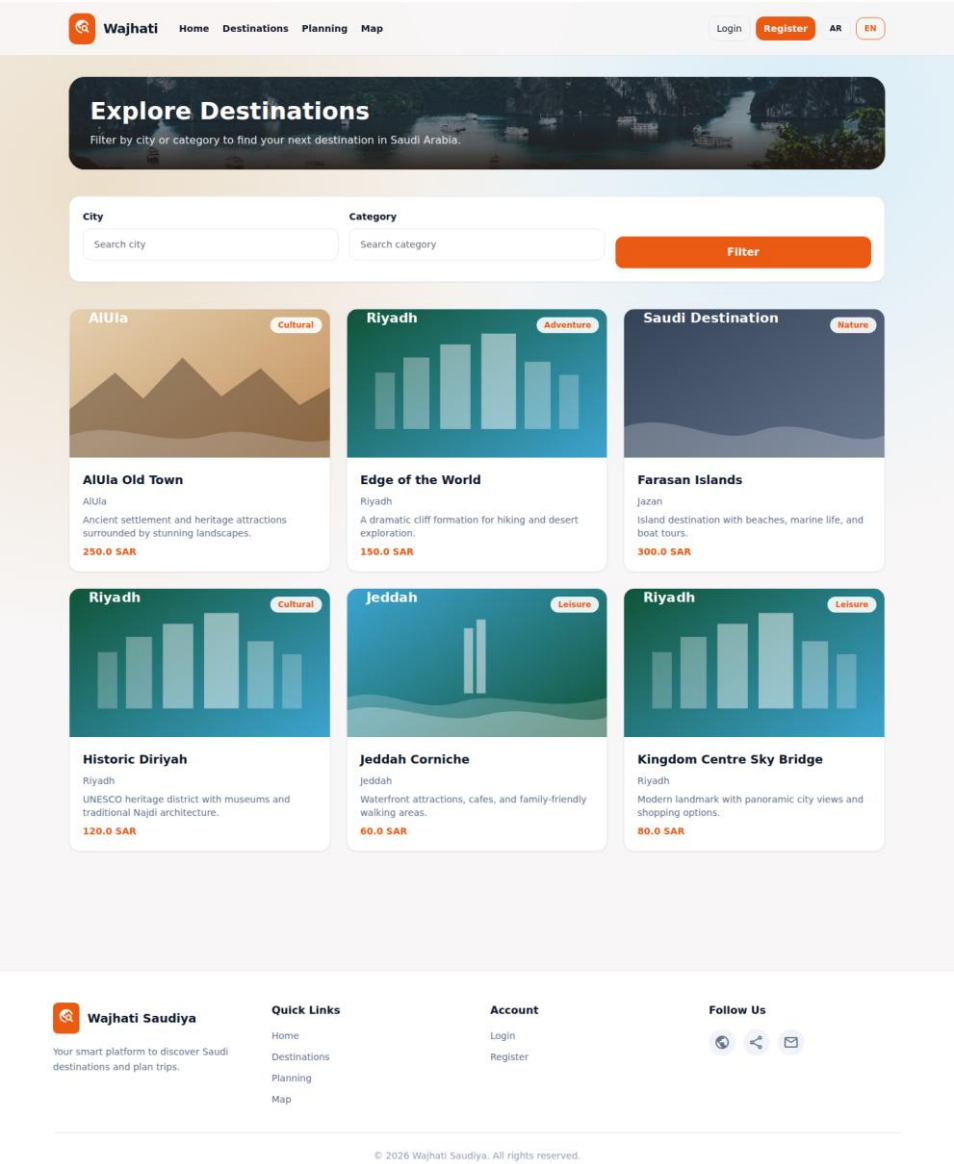
The project is successful because it reaches functional completeness within its scope. It does not attempt to solve every tourism technology problem. Instead, it solves the narrower but important problem of helping users explore Saudi destinations and transform preferences into actionable trip plans. That is a strong and defensible contribution for an applied diploma project.

Figure 5.1 Live public home interface



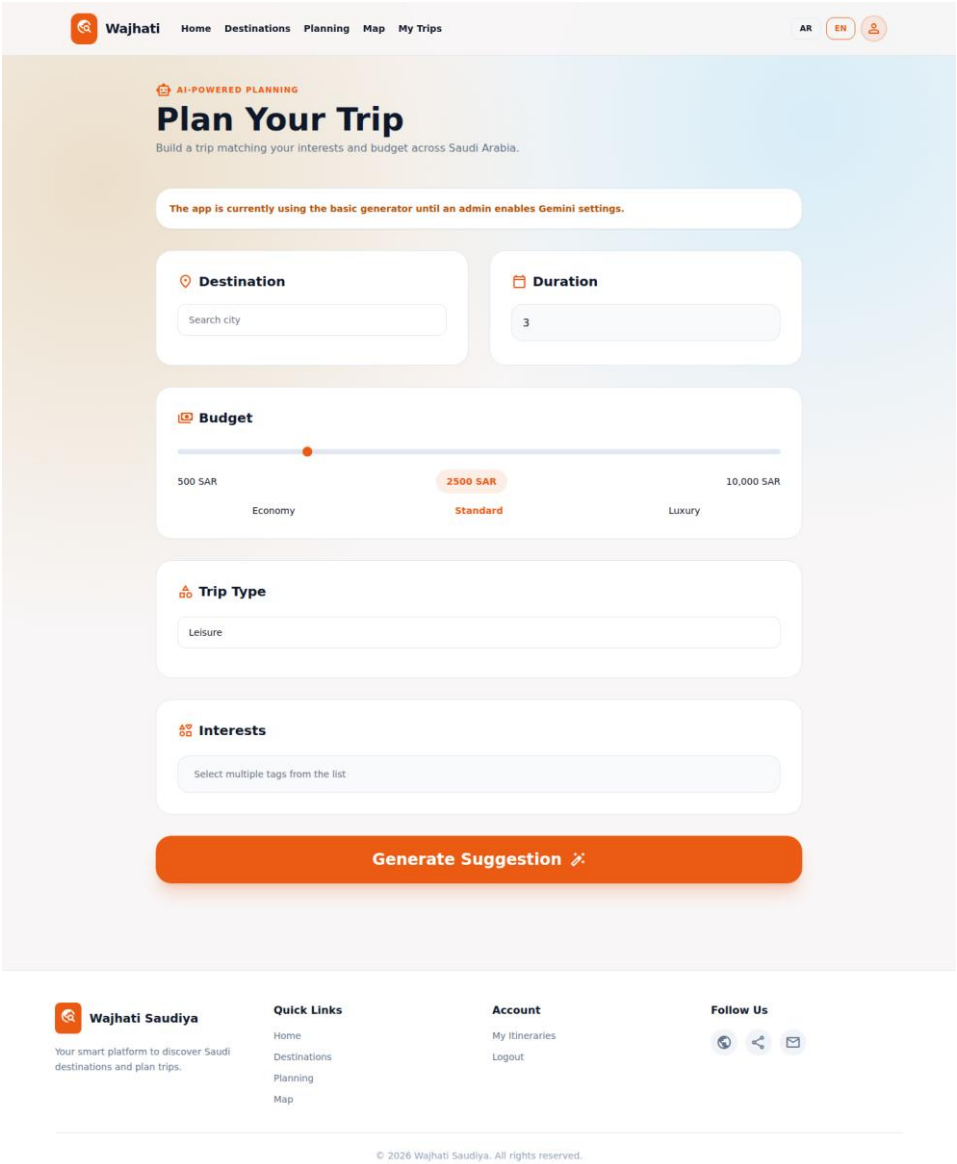
Live public home page supports the overall usability story of the application by giving the user a clear visual step in the travel-planning journey. In the context of this report, the screen also acts as evidence that the implemented system is operational at the interface level and not merely theoretical.

Figure 5.2 Live destinations browsing interface



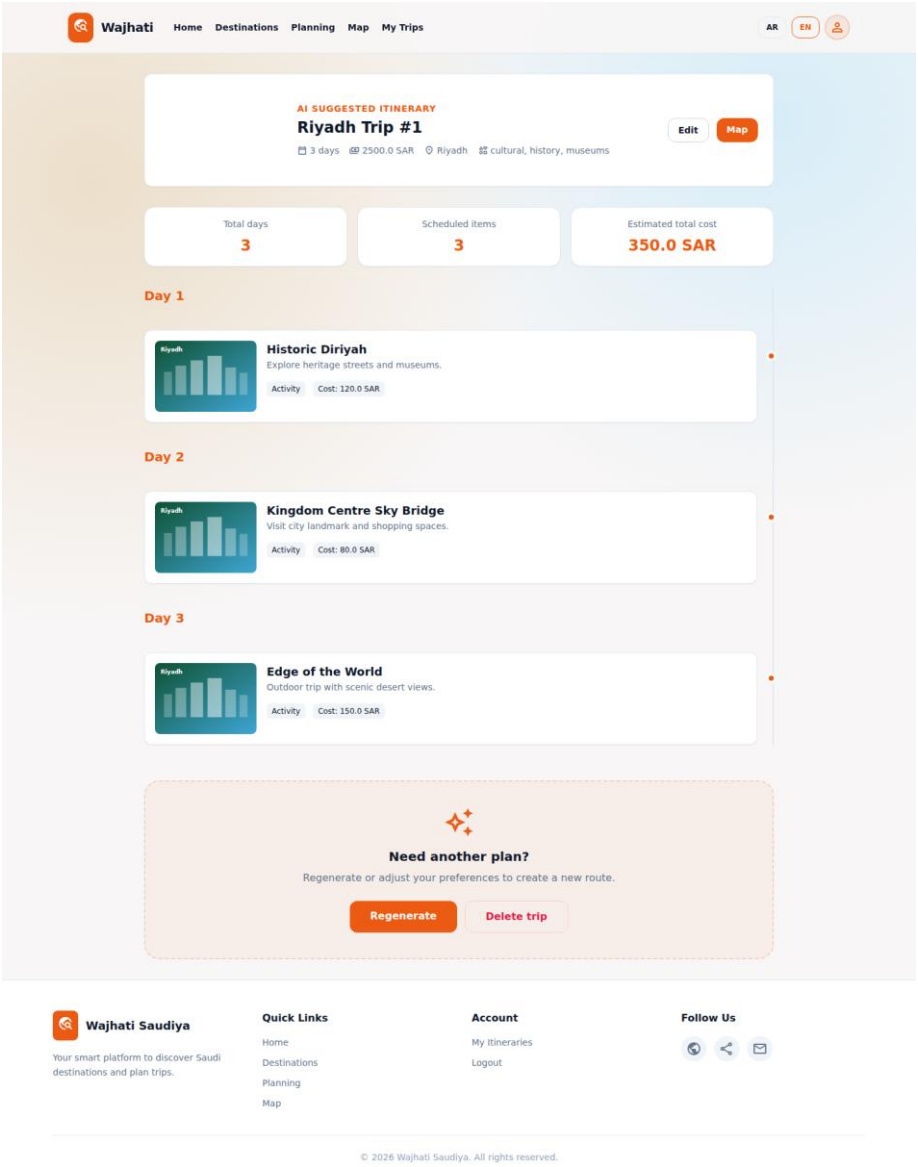
Live destinations page supports the overall usability story of the application by giving the user a clear visual step in the travel-planning journey. In the context of this report, the screen also acts as evidence that the implemented system is operational at the interface level and not merely theoretical.

Figure 5.3 Live itinerary creation interface



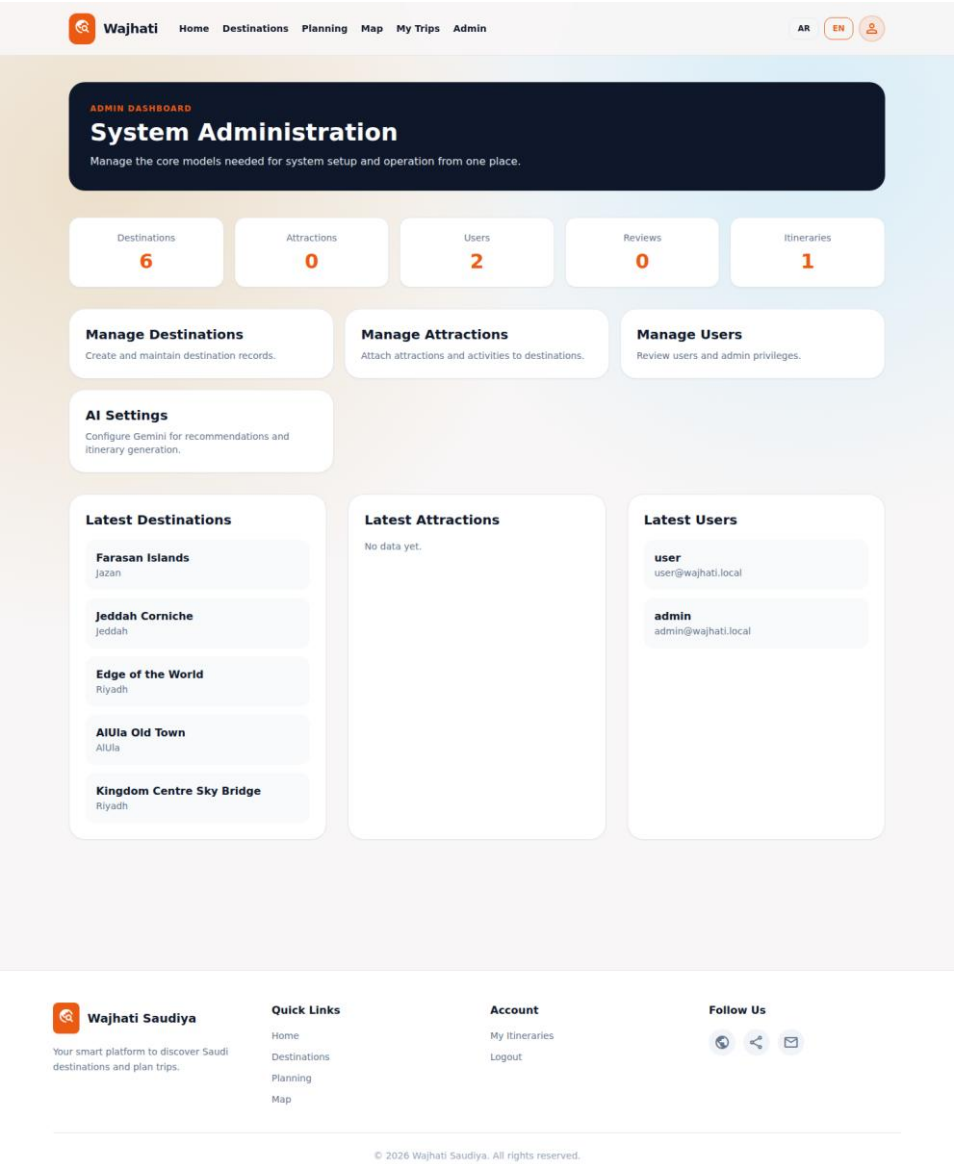
Live itinerary creation page supports the overall usability story of the application by giving the user a clear visual step in the travel-planning journey. In the context of this report, the screen also acts as evidence that the implemented system is operational at the interface level and not merely theoretical.

Figure 5.4 Live itinerary detail interface



Live itinerary detail page supports the overall usability story of the application by giving the user a clear visual step in the travel-planning journey. In the context of this report, the screen also acts as evidence that the implemented system is operational at the interface level and not merely theoretical.

Figure 5.5 Live admin dashboard interface



Live admin dashboard supports the overall usability story of the application by giving the user a clear visual step in the travel-planning journey. In the context of this report, the screen also acts as evidence that the implemented system is operational at the interface level and not merely theoretical.

Table 5.1 Testing and Evaluation Summary

Area	Observed Strength	Remaining Gap
Authentication	Clear login and registration workflow with password hashing.	Needs broader negative-case and authorization tests.
Recommendation	Core matching and itinerary generation covered by tests.	More edge cases and budget scenarios can be added.
Persistence	Trips, favorites, and reviews are stored relationally.	No migration workflow yet.
Administration	Runtime settings and content management are represented in the app.	Would benefit from deeper admin audit and form tests.
API	Provides reusable JSON endpoints for core planning features.	Could expand documentation and error-case coverage.

Chapter 6

Database Design

6.1 Database Design Overview

The database design of Wajhati Saudiya is based on a relational structure implemented through SQLAlchemy models and backed by SQLite in the current development environment. This design is appropriate for the project because it separates user identity, tourism content, user interaction, recommendation output, and system configuration into clearly defined tables.

A good academic database design should reduce duplication, preserve relationship clarity, and support realistic queries. The current schema satisfies these principles by isolating main subjects such as users, destinations, attractions, itineraries, favorites, reviews, and application settings instead of mixing them into a small number of overloaded tables.

6.2 Entity and Attribute Design Rationale

The `User` entity stores identity, access, and personalization attributes. The `Destination` entity stores the core tourism record for each location, including city, category, descriptive text, estimated cost, and coordinates. `Attraction` extends the destination model by describing activity-level entries that belong to destinations.

The `Itinerary` and `ItineraryItem` entities form a parent-child structure that is especially important in the project. This separation allows a trip request to be stored once while preserving multiple day-level entries underneath it. The result is cleaner than storing all itinerary content in a single text field and it supports later reporting or interface grouping by day number.

The `Favorite` and `Review` entities are interaction tables that link users to destinations. Their separation is appropriate because they represent different business meanings: a favorite is a

bookmark or preference indicator, while a review is evaluative feedback. Finally, `AppSetting` supports configuration values that are not naturally owned by a single user or destination row.

6.3 Relationship Design

The schema uses one-to-many relationships where ownership is clear. One user can own many itineraries, reviews, and favorites. One destination can own many attractions and reviews. One itinerary can own many itinerary items. This is consistent with the actual use cases implemented by the application.

The design also includes a uniqueness rule on the favorite relationship so that one user cannot create duplicate favorite links to the same destination. This is a useful example of a business rule enforced at the data layer rather than left entirely to interface behavior.

These relationships are academically strong because they can be mapped directly to the system functions documented in the report. The user saves an itinerary because a `User` has many `Itinerary` objects. The user reviews a destination because `Review` links the two entities. This traceability makes the schema easy to justify in front of a committee.

6.4 Evaluation of the Current Database Choice

SQLite is a reasonable choice for the current stage of the project because it simplifies local development, testing, and demonstration. It does not require a separate database server and makes the application easier to run in a university lab or student laptop environment.

However, SQLite is not the ideal long-term choice for a larger production platform with concurrent users, stronger administration requirements, and more advanced operational controls.

For that reason, the current database should be described as appropriate for prototype deployment rather than final large-scale release.

The current schema itself is reusable beyond SQLite. If the project is extended later, the model definitions can be migrated to a stronger database backend with minimal conceptual redesign, provided that a migration workflow is introduced.

6.5 Database Design Summary

Overall, the database design is suitable for an applied diploma project because it is normalized enough to support realistic workflows without becoming unnecessarily complex. It shows that the project team understood how to translate user requirements into related entities, attributes, constraints, and ownership relationships.

The schema also supports future extension. Additional entities such as bookings, route segments, attachments, analytics, or notification records could be added later without forcing a redesign of the current core tables. That extensibility is a positive design quality.

Table 6.1 Database Design Strengths

Aspect	Strength	Academic Value
Normalization	Main subjects are separated into distinct tables.	Reduces redundancy and improves clarity.
Relationships	User, destination, review, itinerary, and item ownership are explicit.	Supports accurate data modeling discussion.
Constraints	Favorite uniqueness and foreign keys preserve consistency.	Shows business rules at the data layer.
Extensibility	Additional tourism or booking modules can be added later.	Demonstrates future-proof design thinking.

Chapter 7

User Interface and Conclusion

7.1 User Interface Overview

The system includes a complete user interface composed of a home page, login page, registration page, profile page, destinations page, destination detail page, itinerary planning page, itinerary detail page, saved itineraries page, map page, and administration pages.

These screens form a complete workflow rather than isolated demonstrations. The user can enter the system, browse information, create an account, personalize preferences, generate a plan, and preserve trip results for later review. This end-to-end continuity is one of the strongest aspects of the project.

7.2 Main Interface Screens

The home page acts as the main landing area and also serves as a lightweight dashboard when the user is authenticated. The login and registration pages provide straightforward account access and creation. The destinations page supports browsing and filtering, while the detail page extends the experience with reviews and favorite control.

The itinerary planning page is central to the system because it collects user preferences and returns a generated travel suggestion. The preview-confirm-regenerate behavior makes this screen especially strong because it gives users control over generated output instead of forcing automatic storage.

The map page adds a spatial browsing layer, and the administration screens provide governance over destinations, attractions, users, and AI settings. Together, these screens demonstrate that Wajhati Saudiya is a functional information system rather than a static prototype.

7.3 Interface Design Evaluation

The interface design is practical and understandable. It is not overloaded with unnecessary complexity, and the user journey is easy to explain in an academic presentation. The bilingual nature of the interface also improves accessibility and reflects the local context of the project.

One of the most important design decisions is that recommendation output is previewed before it is saved. This increases trust and gives the user a chance to regenerate the result if the first draft does not fully match expectations. This is a good interaction design choice for planning-oriented software.

7.4 Final Conclusion

Wajhati Saudiya is a successful applied project because it delivers a coherent domestic tourism planning workflow with practical technical depth. The project integrates browsing, filtering, account management, favorites, reviews, itinerary generation, persistence, API exposure, and administrative control in a single system.

The application is especially strong as a university project because it demonstrates full-stack thinking. It includes a relational data model, modular route design, rule-based recommendation logic with optional AI enhancement, and enough testing evidence to support evaluation of the critical planning workflow.

Although the project still has limitations such as local-development orientation, a small dataset, and the absence of production deployment features, it already provides a solid

foundation for future enhancement. As a diploma-level information systems project, it is technically credible, practically useful, and well aligned with the stated academic objectives.

Appendix A: API Summary

Endpoint	Method	Purpose
/api/health	GET	Returns a simple service health payload.
/api/destinations	GET	Returns destinations with optional city/category filtering.
/api/itineraries/generate	POST	Generates itinerary output and optionally saves it.
/api/destinations/<id>/reviews	GET	Returns review data for one destination.

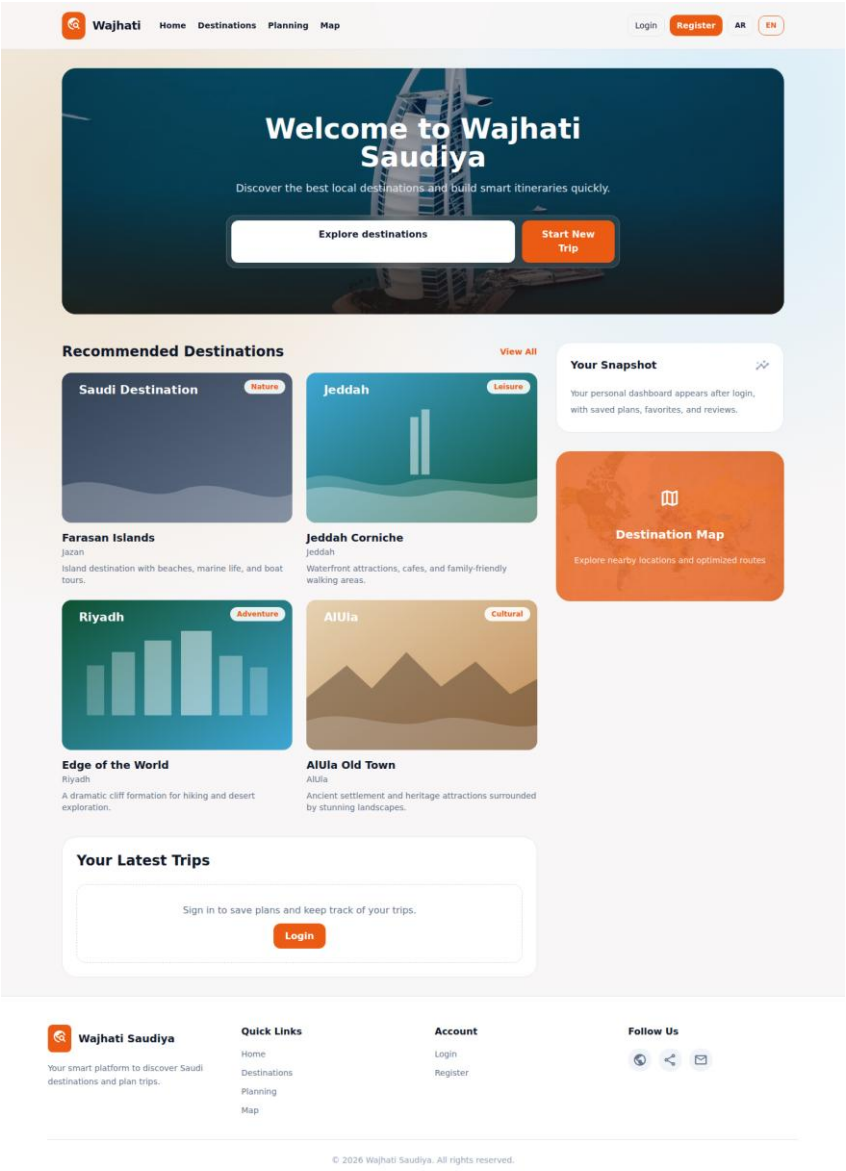
Appendix B: Data Entity Summary

Entity	Key Fields	Explanation
User	email, password_hash, is_admin, preferred_language	Identity, role, and personalization record.
Destination	city, category, estimated_cost, season	Core tourism destination record.
Attraction	destination_id, entry_cost, duration_hours	Sub-activity under a destination.
Itinerary	destination_city, duration_days, budget, interests	Saved trip request and result summary.
ItineraryItem	day_number, title, estimated_cost	One day-level itinerary entry.
Review / Favorite	user_id, destination_id	User interaction links with destinations.

Appendix C: Live Website Interface Images

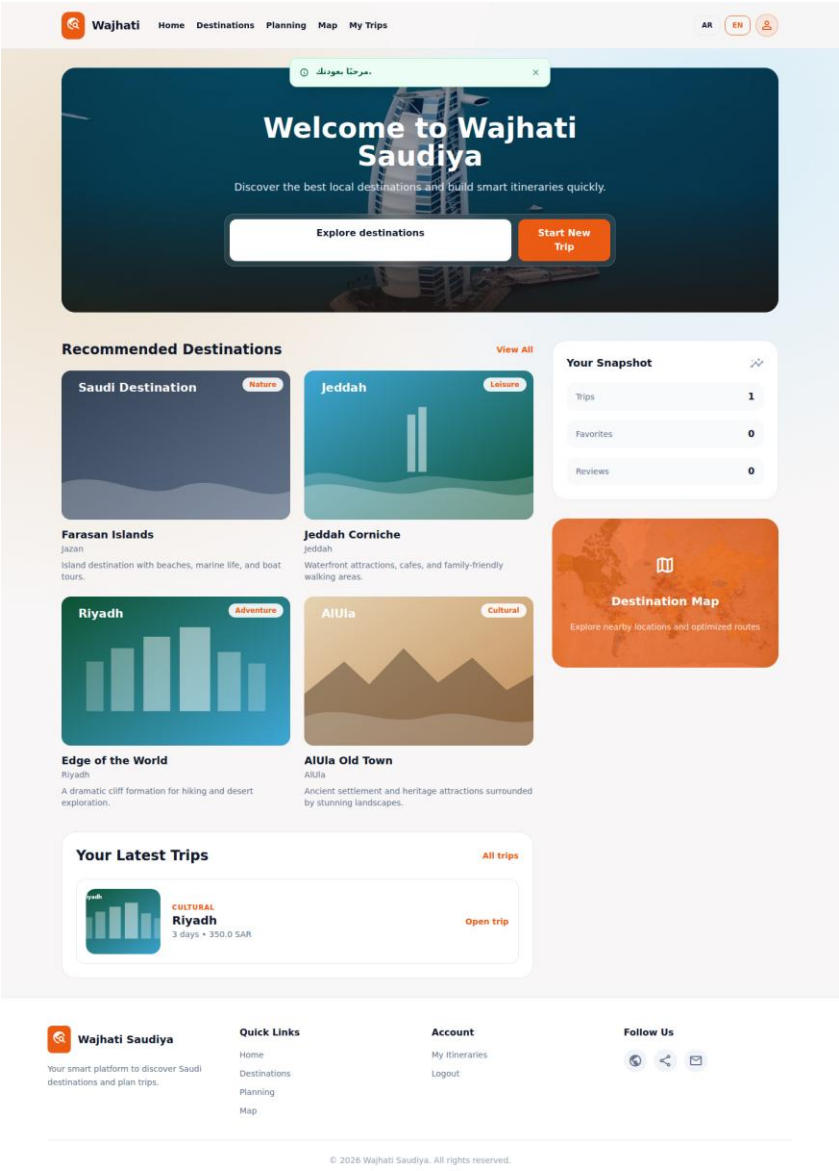
The following figures were captured again from the running website itself using the real project routes and templates. They replace the earlier reused design images and document the actual interface output produced by the current codebase.

Figure C.1 Live public home page



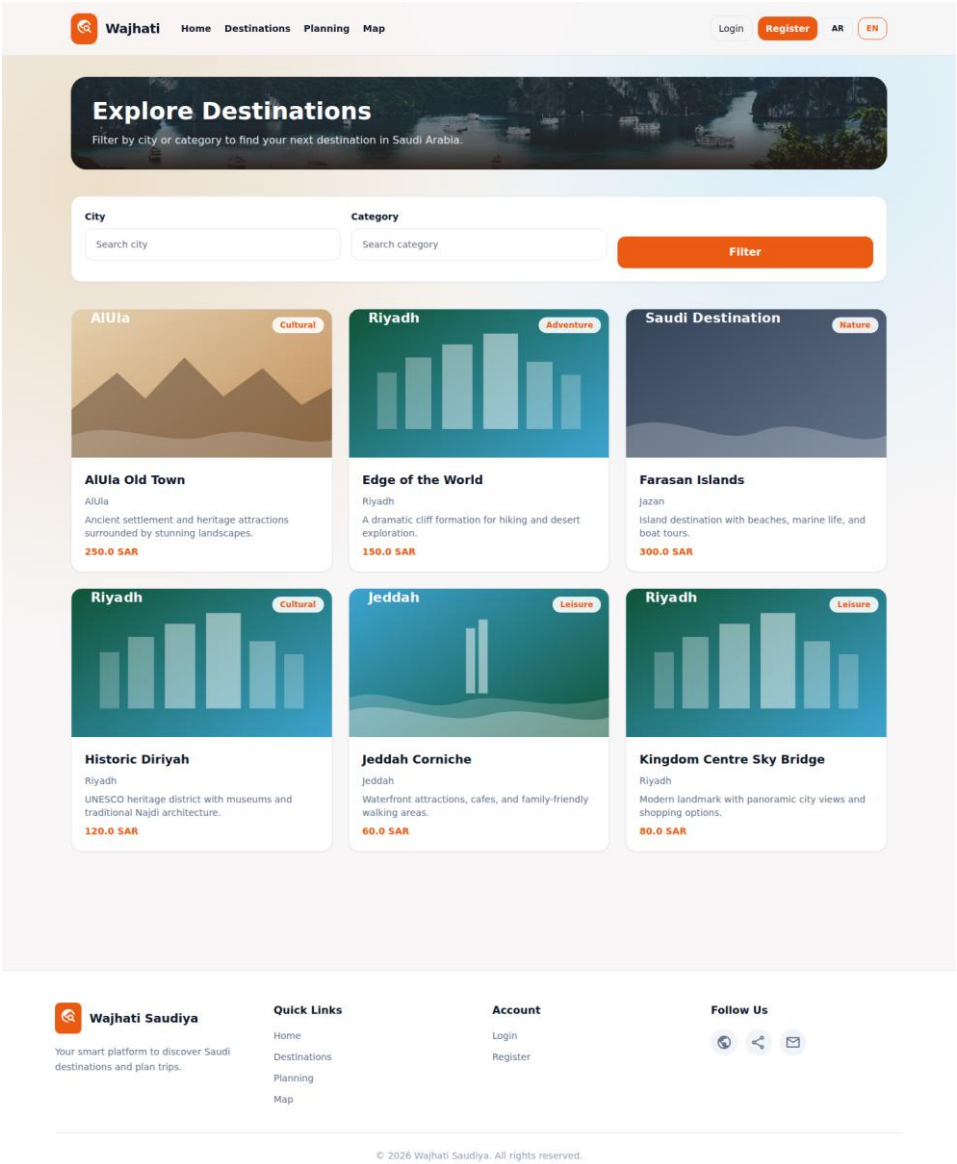
Live public home page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.2 Live user home page



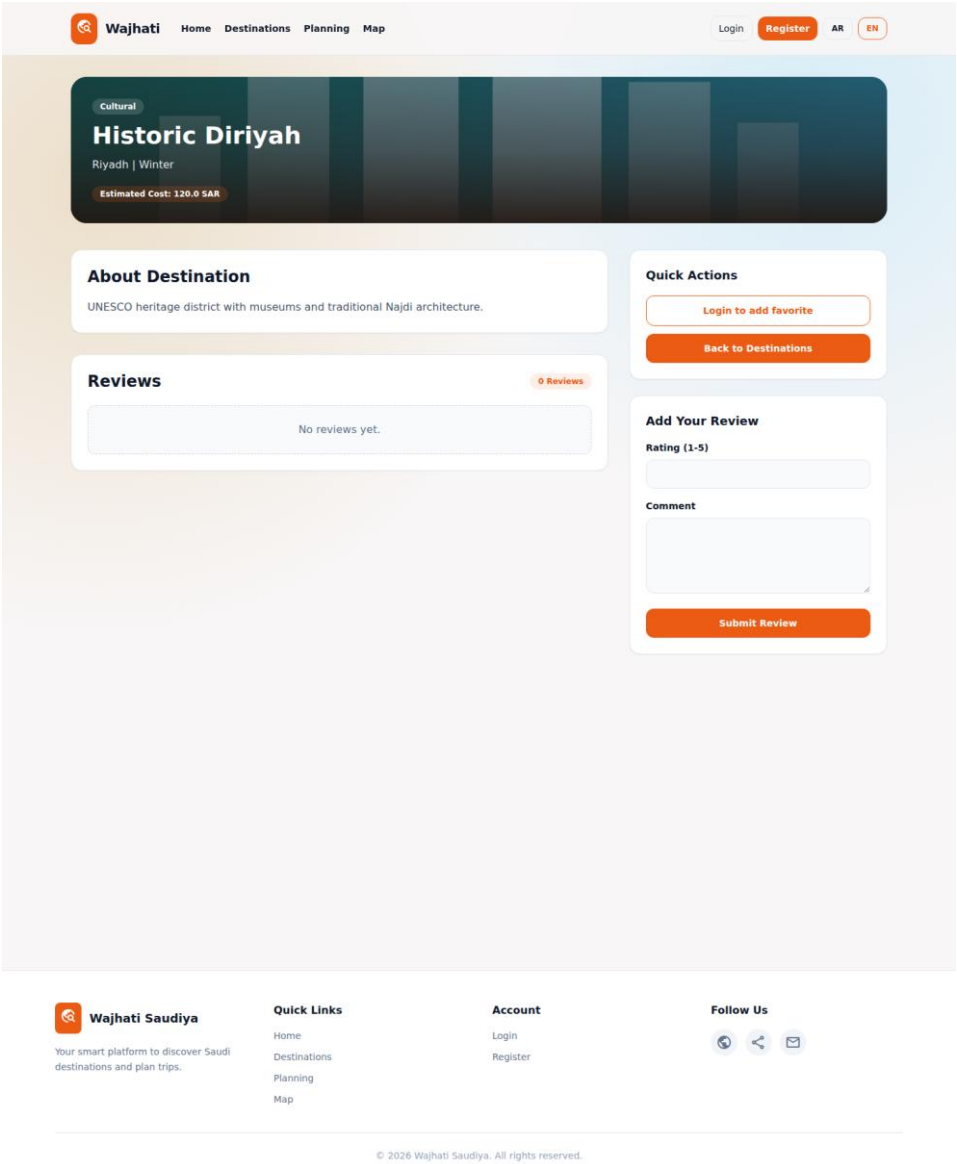
Live user home page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.3 Live destinations page



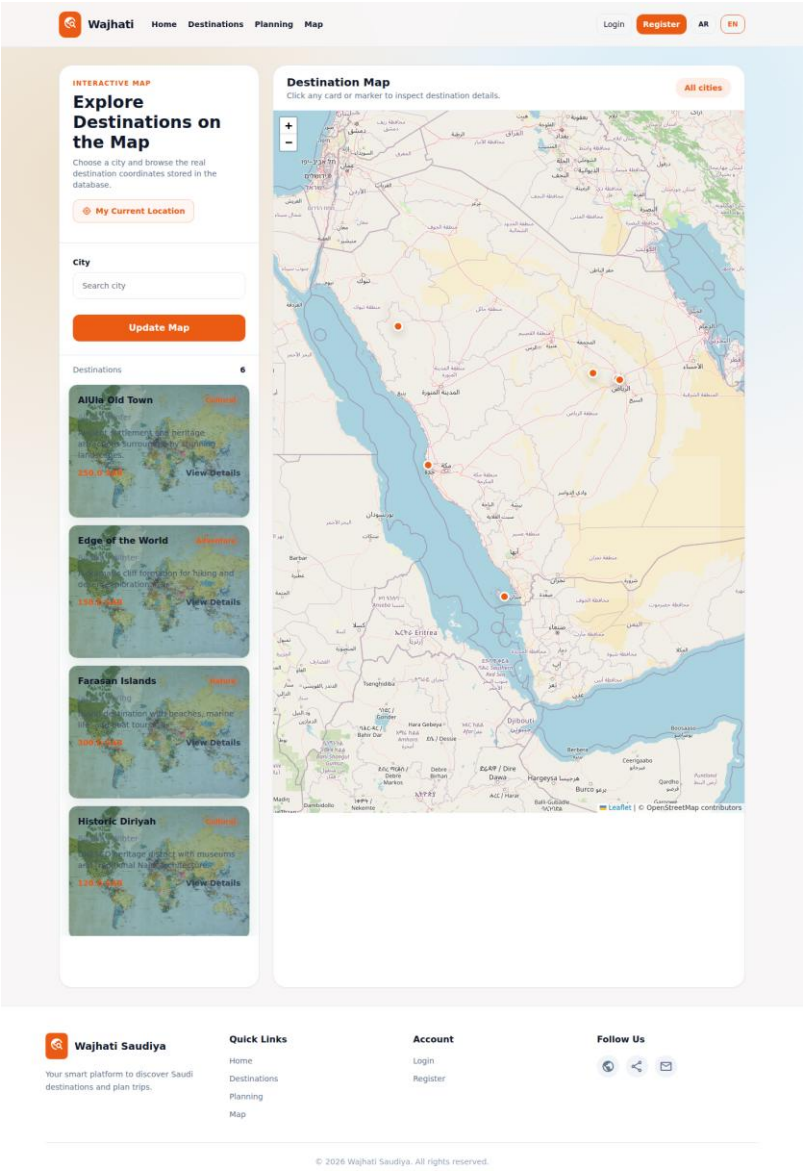
Live destinations page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.4 Live destination detail page



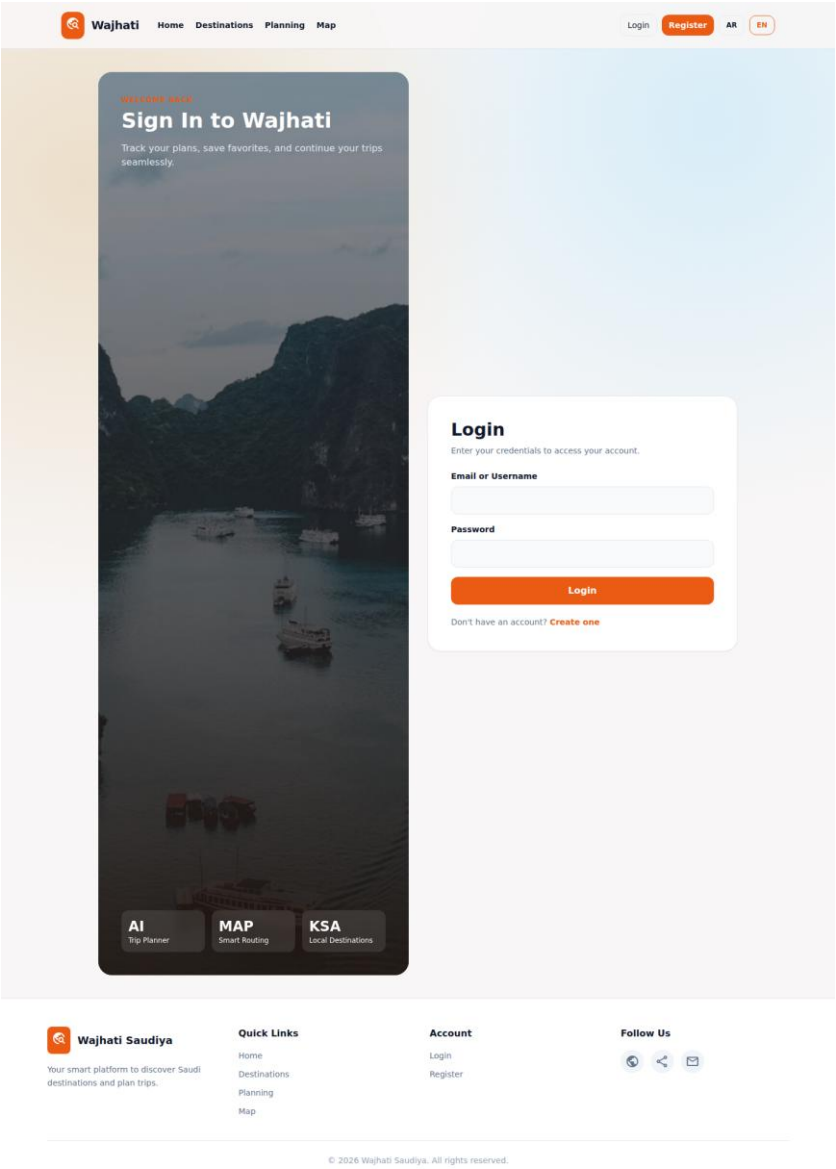
Live destination detail page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.5 Live map page



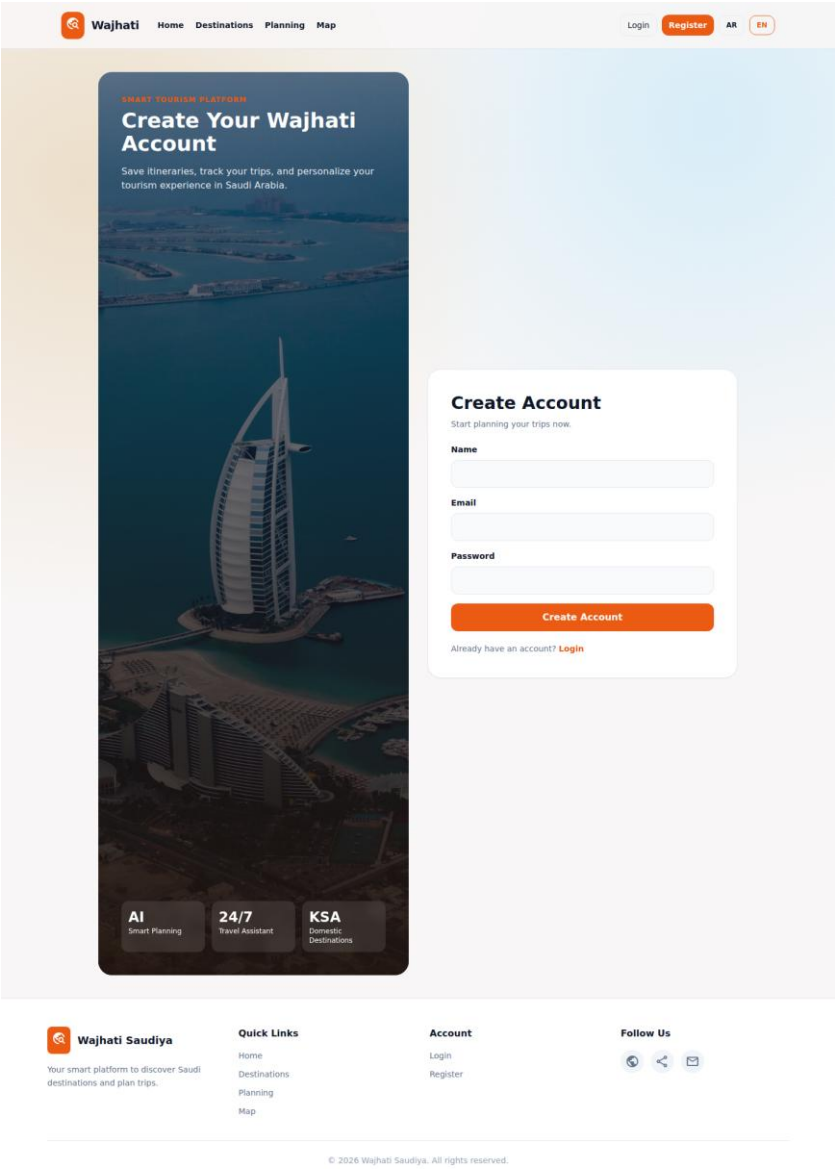
Live map page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.6 Live login page



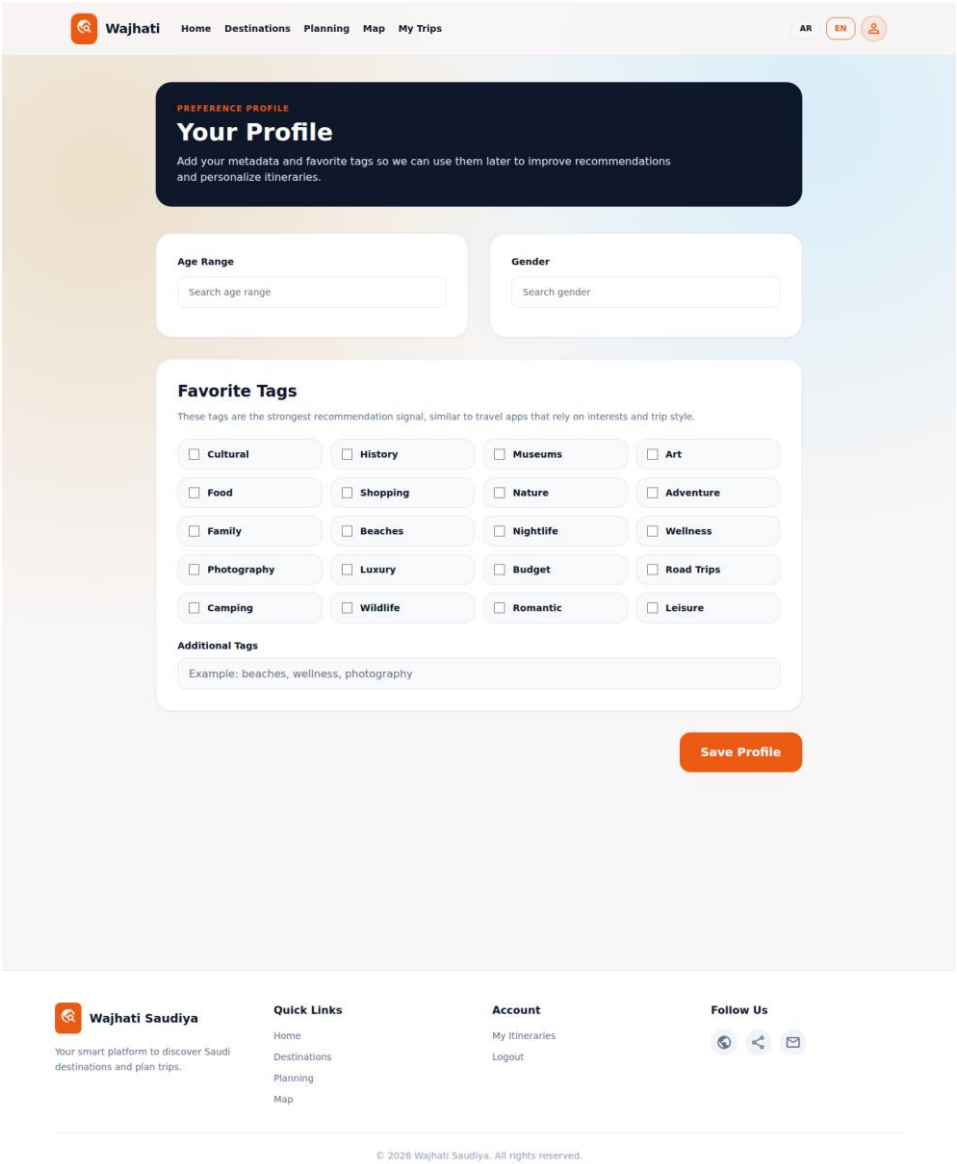
Live login page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.7 Live register page



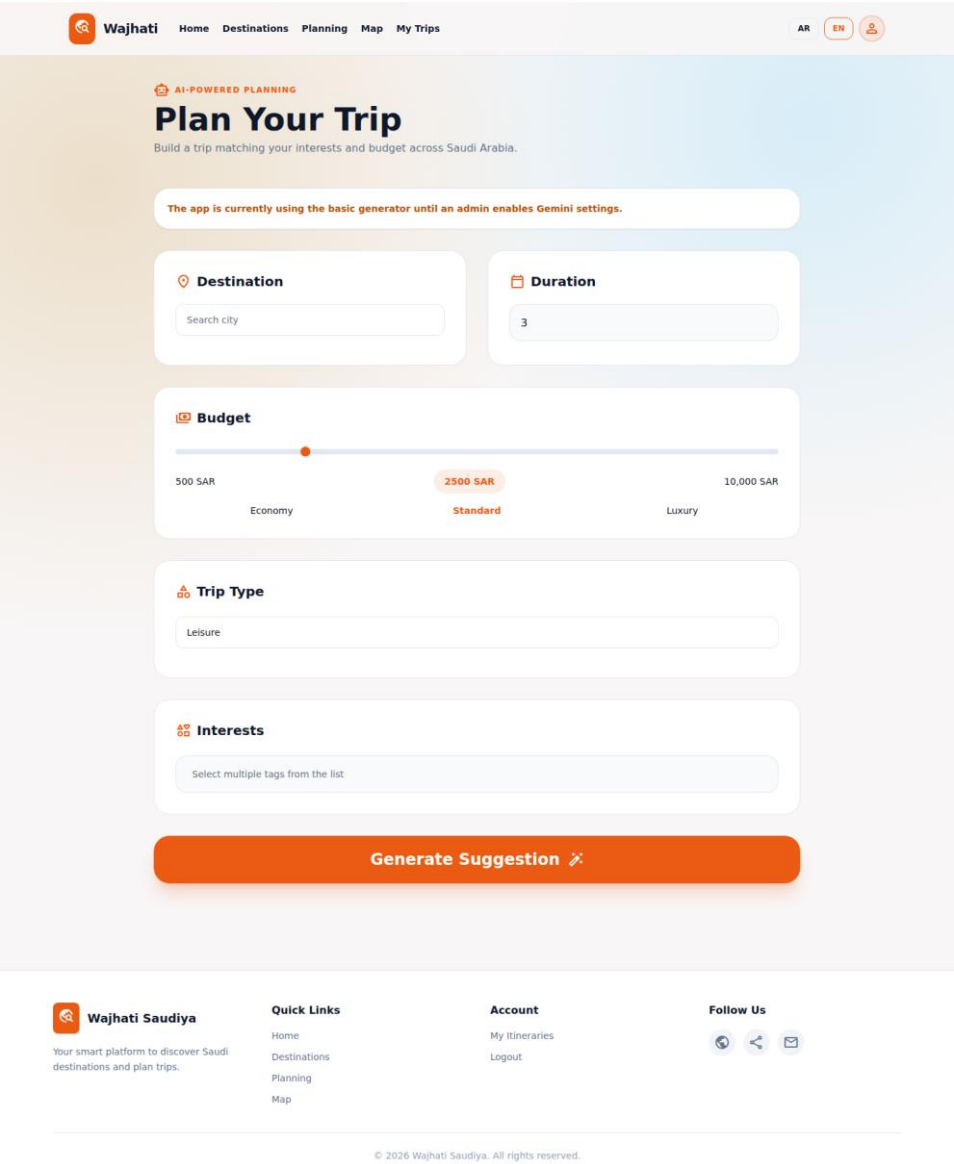
Live register page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.8 Live profile page



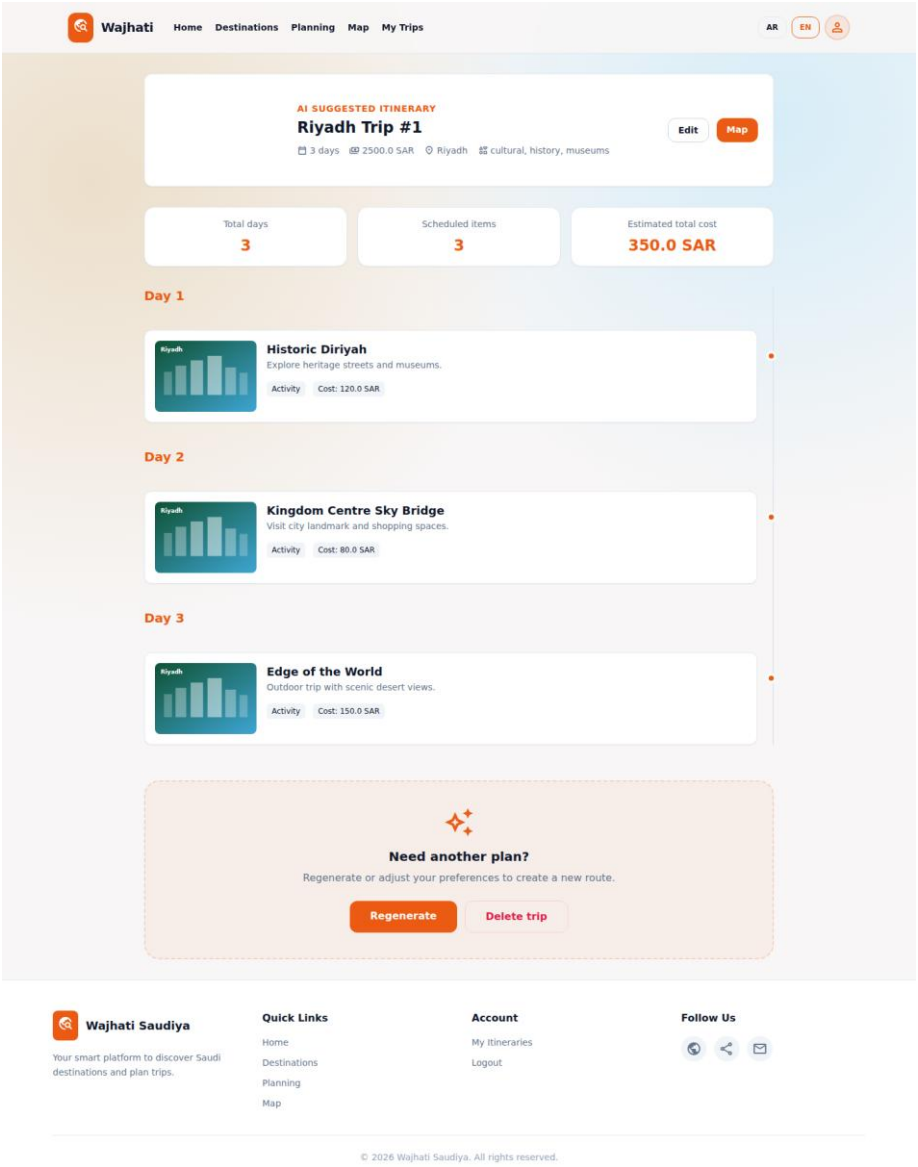
Live profile page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.9 Live itinerary creation page



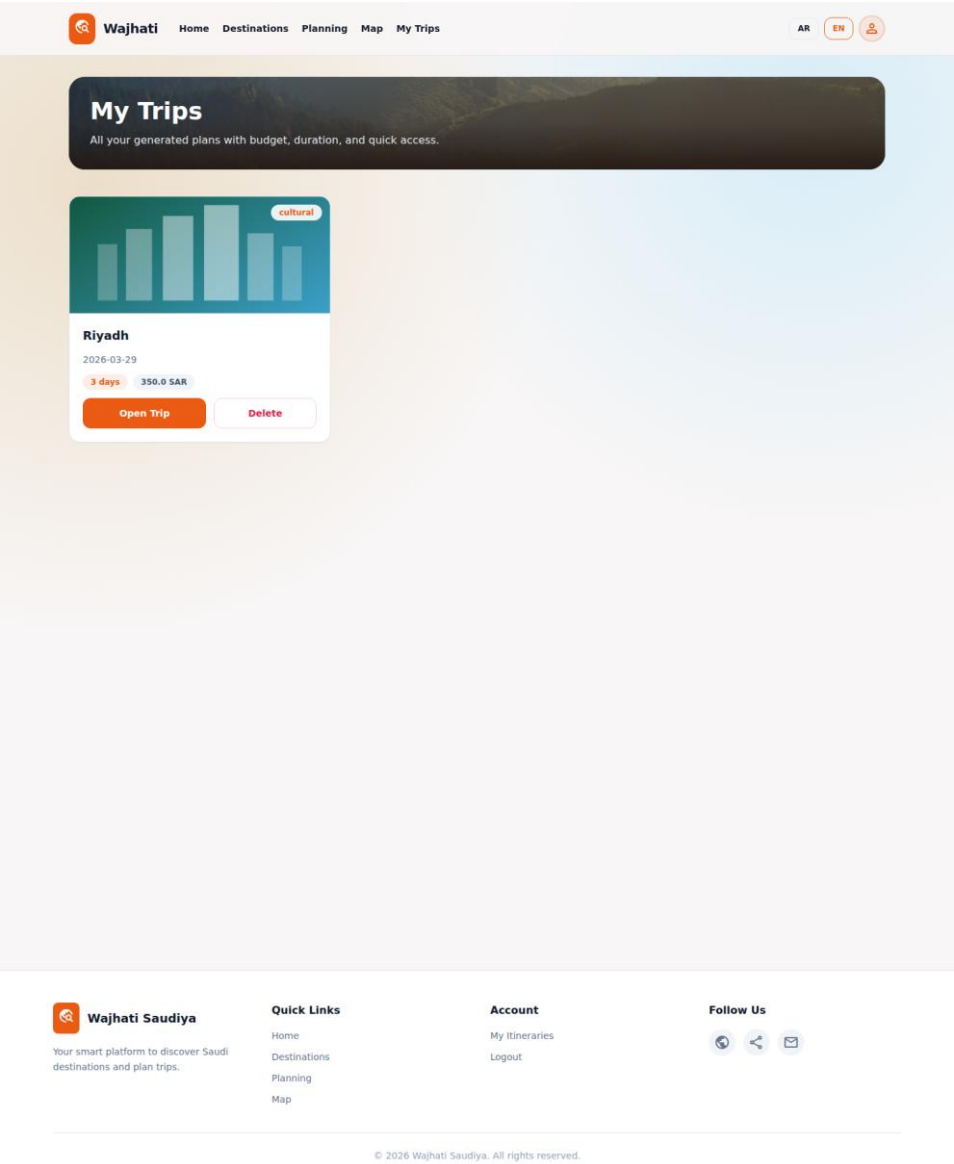
Live itinerary creation page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.10 Live itinerary detail page



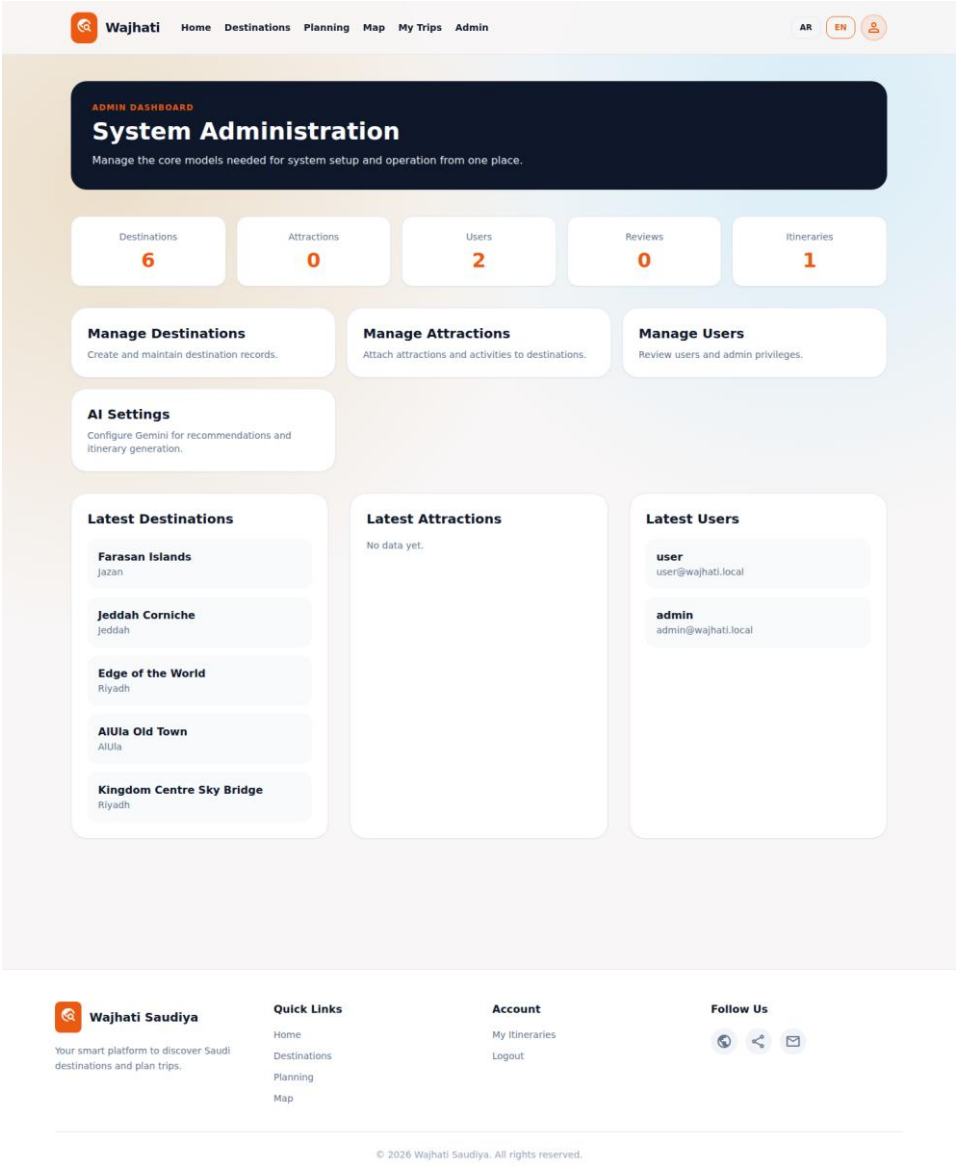
Live itinerary detail page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.11 Live my itineraries page



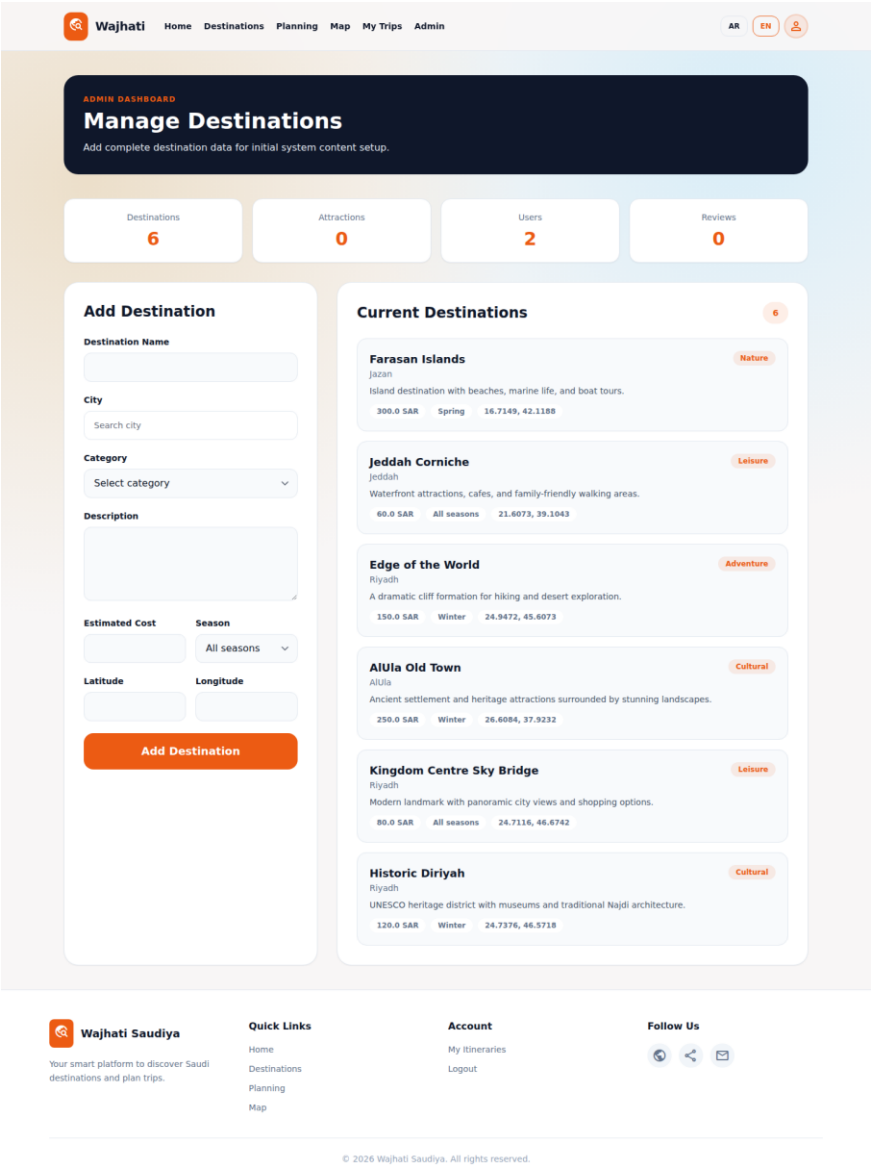
Live my itineraries page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.12 Live admin dashboard



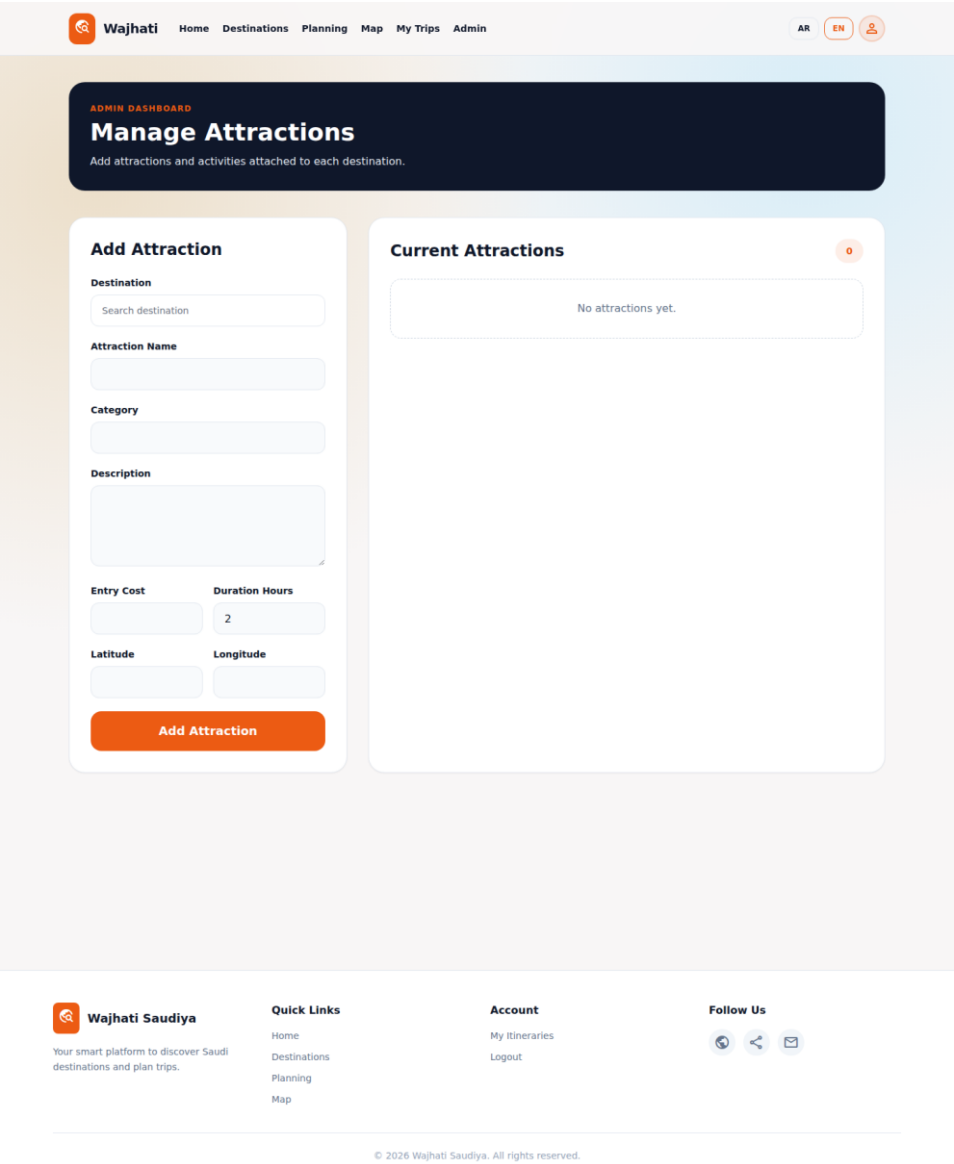
Live admin dashboard was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.13 Live admin destinations page



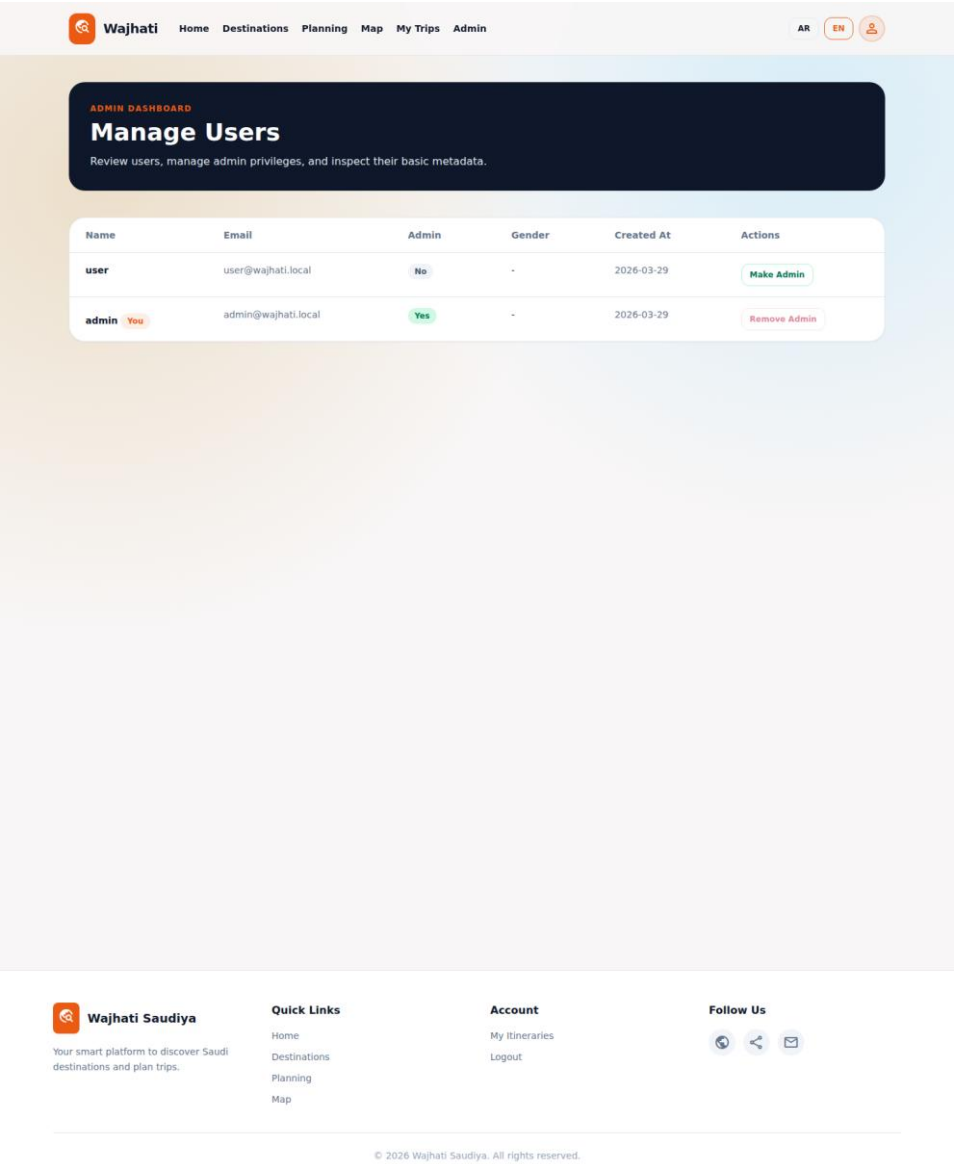
Live admin destinations page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.14 Live admin attractions page



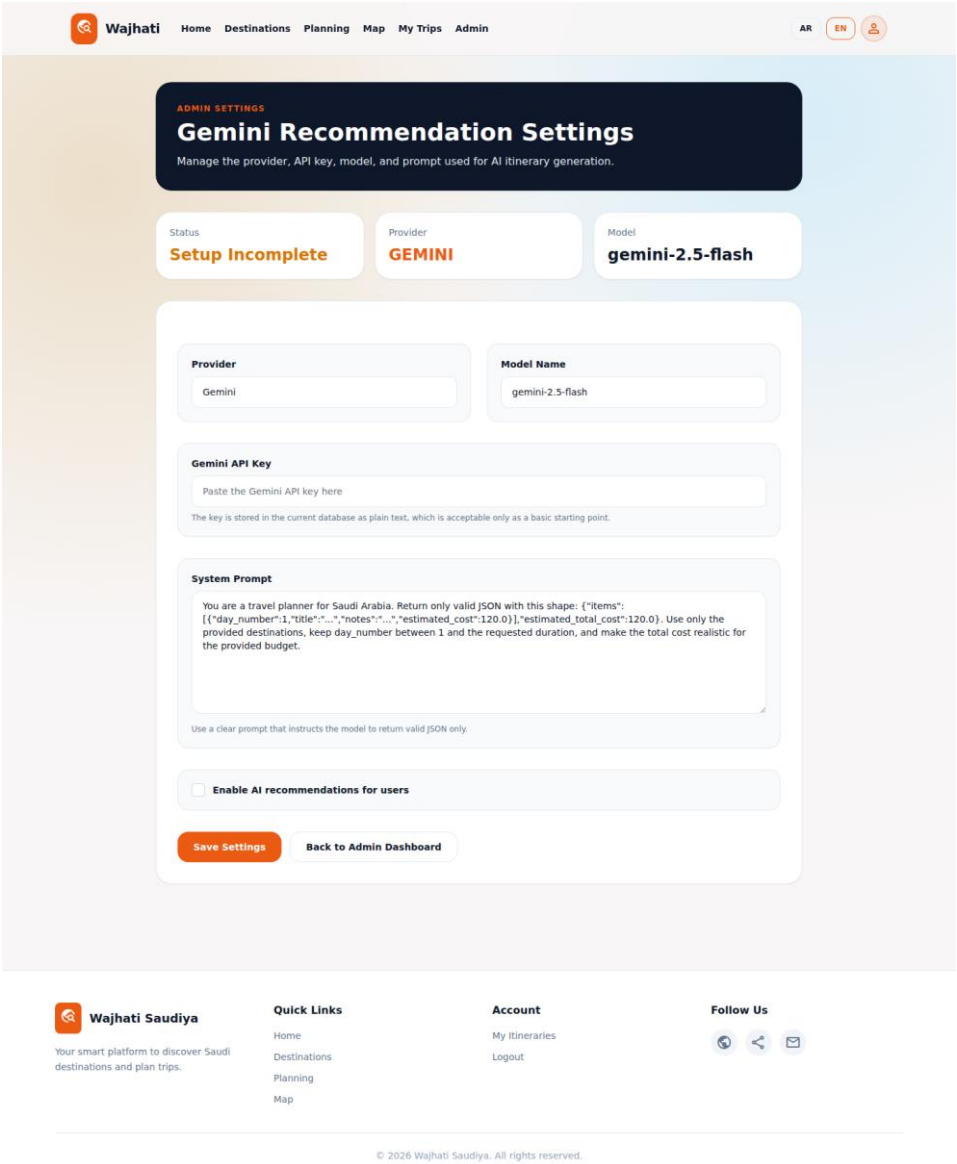
Live admin attractions page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.15 Live admin users page



Live admin users page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Figure C.16 Live admin AI settings page



Live admin AI settings page was captured from the running Flask application and corresponds directly to a page implemented by the project templates and routes.

Appendix D: Important Code Snippets

This appendix includes selected code excerpts that represent the core technical implementation of the application. The snippets are intentionally short and focused on the most important behavior in the project.

D.1 Application Entry Point

```
from wajhati import create_app

app = create_app()


if __name__ == "__main__":
    app.run(debug=True)
```

D.2 Application Factory

```
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)

    db.init_app(app)
    login_manager.init_app(app)

    from wajhati.routes.auth import auth_bp
    from wajhati.routes.main import main_bp
    from wajhati.routes.api import api_bp

    app.register_blueprint(auth_bp)
    app.register_blueprint(main_bp)
    app.register_blueprint(api_bp, url_prefix="/api")

    with app.app_context():
        db.create_all()
        _ensure_user_profile_columns()
        seed_default_users()
        _ensure_admin_user()
        seed_demo_destinations()

    return app
```

D.3 Destination Matching Logic

```
def match_destinations(destinations, city, budget, interests, profile_context=None):
    normalized_city = city.strip().lower()
    normalized_interests = _normalize_text_list(interests)
    normalized_profile = _normalize_profile_context(profile_context)
    normalized_favorite_tags = _normalize_text_list(normalized_profile["favorite_tags"])
    matched = []

    for destination in destinations:
        if normalized_city and destination.city.lower() != normalized_city:
            continue

        score = 0
        score += 3
        if destination.estimated_cost <= budget:
            score += 2
        if destination.category.lower() in normalized_interests:
            score += 2
        if destination.category.lower() in normalized_favorite_tags:
            score += 2

        matched.append((score, destination))

    matched.sort(key=lambda item: (item[0], -item[1].estimated_cost), reverse=True)
    return [destination for score, destination in matched if score > 0]
```

D.4 API Itinerary Generation Endpoint

```
@api_bp.post("/itineraries/generate")
def api_generate_itinerary():
    payload = request.get_json(silent=True) or {}
    city = str(payload.get("destination_city", "")).strip()
    trip_type = str(payload.get("trip_type", "leisure")).strip().lower()
    interests = _parse_interests(payload.get("interests", []))

    duration_days = int(payload.get("duration_days", 1))
    budget = float(payload.get("budget", 0))

    destinations = Destination.query.all()
    profile_context = _current_user_profile_context()
    matched = match_destinations(destinations, city=city, budget=budget, interests=interests,
profile_context=profile_context)

    generated = generate_itinerary(
        matched,
        duration_days,
        budget,
        trip_type,
```

```
        interests,
        profile_context=profile_context,
        ai_settings=get_ai_settings(),
    )

    return jsonify({
        "destination_city": city,
        "duration_days": duration_days,
        "budget": budget,
        "trip_type": trip_type,
        "interests": interests,
            "estimated_total_cost": generated["estimated_total_cost"],
        "items": generated["items"],
    })
```

D.5 AI Settings Access

```
def get_ai_settings():
    return {
        "enabled": AppSetting.get_value("ai_recommendations_enabled", "0") == "1",
        "provider": AppSetting.get_value("ai_recommendations_provider", AI_PROVIDER_GEMINI) or
AI_PROVIDER_GEMINI,
        "model": AppSetting.get_value("ai_recommendations_model", DEFAULT_GEMINI_MODEL) or
DEFAULT_GEMINI_MODEL,
        "api_key": AppSetting.get_value("ai_recommendations_api_key", "").strip(),
        "system_prompt": AppSetting.get_value("ai_recommendations_system_prompt",
DEFAULT_AI_SYSTEM_PROMPT)
        or DEFAULT_AI_SYSTEM_PROMPT,
    }
```